



# Fully Unsupervised Entity Resolution with Fine-Tuned Language Embeddings

## Vollständig automatisierte Duplikaterkennung unter Verwendung nachtrainierter Sprachmodelle

**Florian Sold**

Universitätsmasterarbeit
zur Erlangung des akademischen Grades

Master of Science
*(M. Sc.)*

im Studiengang
IT Systems Engineering

eingereicht am 10. Januar 2025 am
Fachgebiet Informationssysteme der
Digital-Engineering-Fakultät
der Universität Potsdam

**Betreuer** Prof. Dr. Felix Naumann
Prof. Dr. Fabian Panse

# Abstract


> To be labeled, or not to be,
> that is the question.
>
> *Inspired by William Shakespeare*

Entity resolution (or duplicate detection) has been a focal point of research for several decades, with significant progress achieved throughout numerous publications. Despite these advancements, achieving fully automated entity resolution in varying industry contexts remains an unsolved challenge. First, most approaches are developed and benchmarked on a fixed set of "easy" datasets, which lack relevance to practical use cases. Second, dataset-specific optimizations and abstractions negatively impact the generalizability, limiting the applicability of these approaches to unknown datasets. Third and foremost, many state-of-the-art approaches rely on domain knowledge provided by human experts, such as rules or training labels, making them costly due to the limited availability of such expertise.

Although unsupervised approaches offer a potential solution to this challenge, they often fall short of reliably achieving competitive performance with their supervised counterparts across diverse datasets. Here, pre-trained language models offer a strong foundation for truly unsupervised entity resolution approaches, as they inherently capture a broad understanding of various domains. However, their generality often limits performance on domain-specific challenges, where optimized approaches typically deliver higher and more consistent performance.

This thesis proposes a pipeline for unsupervised entity resolution that unites generic pre-trained language models and traditional similarity measures. Through domain-specific fine-tuning of pre-trained language models with weak labels derived from these measures, the pipeline integrates both knowledge sources to adapt effectively to the target domain and achieve more competitive matching performance. We begin by reviewing the state-of-the-art in entity resolution and introducing the necessary technological foundations. Subsequently, we present our pipeline and thoroughly analyze its core components. Finally, we evaluate the performance of our approach relative to prior work and conclude by assessing whether unsupervised entity resolution has become a viable option.

# Zusammenfassung

Duplikaterkennung war in den vergangenen Jahrzehnten Gegenstand zahlreicher Forschungsprojekte. Trotz kontinuierlicher Fortschritte ist es bislang nicht gelungen, die Duplikaterkennung vollständig zu automatisieren und gleichzeitig so abzubilden, dass sie in verschiedenen Praxiskontexten adäquate und effektive Verwendung finden kann.

Erstens sind viele der bisherigen Ansätze speziell für eine kleine Menge einfacher Testdatensätze optimiert, die wenig Bezug zu späteren praktischen Anwendungsszenarien aufweisen. Weiterhin behindern datensatzspezifische Anpassungen die Generalisierbarkeit und damit den Einsatz auf neuen, unbekannten Datensätzen. Schließlich basieren viele der führenden Ansätze auf umfangreichem Domänenwissen, das von menschlichen Experten bereitgestellt werden muss – beispielsweise als umfangreiche Regelsätze oder Trainingslabels – und kostspielig werden kann.

Die vollständig automatisierte Duplikaterkennung stellt eine potenzielle Lösung für diese Probleme dar. Bestehende Ansätze bleiben jedoch häufig deutlich hinter den Erwartungen zurück und erzielen merklich schwächere Ergebnisse als ihre mit Domänenwissen ausgestattete Konkurrenz. Vortrainierte Sprachmodelle verfügen bereits über ein umfangreiches Verständnis verschiedener Domänen und bieten daher einen vielversprechenden Ausgangspunkt für die automatisierte Duplikaterkennung. Ihre Generalität schränkt jedoch im Vergleich zu spezialisierten Ansätzen die Qualität ihrer Ergebnisse deutlich ein.

Um diese Herausforderung zu lösen, stellen wir in dieser Arbeit einen ganzheitlichen Prozess zur automatisierten Duplikaterkennung vor. Dabei kombinieren wir generische, vortrainierte Sprachmodelle mit traditionellen Ähnlichkeitsmaßen, indem wir letztere als ungenaue Labels im Rahmen eines Nachtrainings des Sprachmodells einsetzen. Durch die Kombination beider Wissensquellen kann sich unser Ansatz optimal an die Zieldomäne anpassen und die Ergebnisse in der Duplikaterkennung signifikant verbessern.

Zu Beginn dieser Arbeit befassen wir uns mit bestehenden Ansätzen zur Duplikaterkennung und erklären die technischen Grundlagen unseres Ansatzes. Anschließend führen wir diesen genauer ein und beleuchten jede Komponente unseres Prozesses im Detail. Abschließend setzen wir unsere Ergebnisse in Relation zu bestehenden Ansätzen und bewerten kritisch, wie gut sich unser Prozess für die automatisierte Duplikaterkennung im Praxiskontext eignet.

# Contents

# 1 Tackling Data Quality: The Role of Entity Resolution (ER)

As early as 2011, Peter Sondergaard, then Senior Vice President at Gartner Inc., remarked, "[i]nformation is the oil of the 21st century, and analytics is the combustion engine"[1]. Over the years, numerous reports have validated his perspective, demonstrating that companies must take data-driven decisions to remain competitive [Ahl19; Pat22]. Consequently, an increasing number of departments within companies now engage with data daily and are beginning to tap into (Gen-)AI as a powerful processing tool [McK24]. However, a foundational issue is still not well addressed: automating the data cleaning process to ensure that subsequent data-driven decisions are both accurate and meaningful.

A critical prerequisite for effective data-driven decision-making is clean and reliable data. However, dirty data remains one of the most significant challenges faced by organizations world-wide. Studies by Gartner, MIT Sloan Management Review, and InterSystems/Vitreous World highlight the substantial impact of "bad" data on business revenue, and stress low confidence among business leaders in the reliability of their data [RD23].

The solution to this challenge is well-established: data cleaning. However, organizations continue to face significant barriers, including data privacy concerns, insufficient expertise, and difficulties in accessing and preparing data. A study by *S&P Global* among financial services companies [Pat22] identifies these challenges as some of the most critical obstacles to becoming more data-driven. Most relevant to this thesis, data-driven decision-making remains hindered by the lack of effective tools for data preparation and cleaning.

Traditionally, data cleaning involves a dedicated pipeline, where the removal of (fuzzy) duplicate entries is a critical step [HN20]. Consider the following example: A corporation operating across multiple countries seeks to aggregate sales data from its subsidiaries. Since some of its business partners also operate in multiple regions, the company must consolidate sales data while matching information about these business partners across various data sources. A simple solution might be to match business partners based solely on the exact equivalence of their names. However, due to regional variations, human errors, and other factors, a single business partner may appear in different forms across data sources. The process of

1 Quote from a speech given at the Gartner Symposium/ITxpo in October 2011 in Florida, U.S.

identifying such "fuzzy" duplicate entries is formally known as *Entity Resolution (ER)* and can be performed on a single dataset or among multiple data sources.

*Entity Resolution* has been subject to research for several decades [Chr12]. In addition to academic efforts and advancements, various commercial solutions have been developed, too. However, existing approaches often rely on domain knowledge supplied as a comprehensive rule set or labeled training data. The former requires domain experts to develop tailored rules for the specific data sources. The latter depends on a reliable source of annotated data, usually collected from proficient human annotators. Both of these approaches incur significant costs, limiting their applicability for many companies.

A potential solution to this problem lies in *Unsupervised Entity Resolution* frameworks. Unlike the methods stated above, these approaches solely utilize the information value found within the data sources, eliminating the need for external domain knowledge. Consequently, unsupervised ER represents an ideal solution for enabling non-experts to perform entity resolution across diverse scenarios – and specifically without any additional dependencies or costs. However, while unsupervised ER approaches do exist, they typically deliver significantly inferior performance compared to their domain knowledge-driven counterparts. In particular, they struggle to provide consistent performance across diverse scenarios, limiting their practical applicability.

The emergence of pre-trained language models (LMs) has introduced a new resource for developing ER frameworks. Consequently, research has increasingly focused on creating LM-based ER approaches, particularly for cases in which labeled training data is available, allowing LMs to be tailored to the target domain [Li+20]. In contrast, only limited research has explored the application of pre-trained language models to unsupervised ER.

Within the broad spectrum of language models, embedding models are of particular interest: Given an input word or sentence, embedding models generate an embedding vector, where semantically similar text yield vectors with high similarity. Research by Zeakis et al. [Zea+23a] has demonstrated promising, though scenario-dependant, performance when applying embedding models to unsupervised entity resolution.

This thesis proposes a novel pipeline for automatically generating "noisy" labels and fine-tuning pre-trained embedding models for unsupervised entity resolution. More specifically, this work seeks to harness the knowledge from traditional similarity measures to create weak labels, enabling improved performance of language models without relying on human expertise or manual annotations. In essence, this approach combines the proven effectiveness of similarity measures with the natural language understanding of language models. Our goal is to develop a

pipeline capable of delivering robust performance across diverse scenarios, making it practical for real-world applications.

## 1.1 Thesis Overview: Structure & Key Topics

The core of this thesis is the presentation of our pipeline and the evaluation of the fine-tuned embedding language model against the baseline results reported by Zeakis et al. [Zea+23a] in Part II.

To lay the necessary foundations, Part I focuses on introducing the necessary concepts and technologies. Chapter 2 formally defines *Entity Resolution* and the notation used throughout this thesis. Subsequently, we introduce various existing approaches, particularly also for unsupervised ER, and explain their characteristics that shape the problem space of this thesis. As a result, this chapter also includes our main related work in the space of ER. The following Chapter 3 focuses on language models and the technological buildings relevant to this thesis. We cover language modeling, embedding models, pre-training procedures, weak labels as well as the fine-tuning process and close by connecting the space of entity resolution to embedding models.

In the beginning of Part II, specifically Chapter 4, we delve deeper into this connection and provide an overview of the "Embeddings4ER" paper by Zeakis et al. [Zea+23a]. The chapter thoroughly explains their core findings and methodology to motivate our opportunity area: to improve the performance of embedding models with fine-tuning. A detailed description and visualization of our pipeline conclude this overview.

Afterward, we focus on each stage of the weak label generation process individually: feature creation from similarity measures (Section 5.1), thresholding (Section 5.2), aggregation (Section 5.3) and finally sampling of training instances (Section 5.4). Each stage includes several methods, particularly also entirely novel ones, and an evaluation by which we select one method for further use. Throughout these stages, we stress the need for generality, enabling our pipeline and the weak label generation process to adapt to various scenarios automatically.

Chapter 6 is dedicated to the fine-tuning process. Here, we begin with an overview of the available hyperparameters and introduce the learning objectives we evaluated. Afterward, we provide results on the performance of the entire pipeline by the matching performance of the fine-tuned embedding model (see Section 6.2).

The thesis concludes by reflecting on its core objective: developing a framework for truly unsupervised entity resolution applicable across various domains. In

Chapter 7, we provide a comprehensive review of our contributions and identify opportunities for future research.

## 1.2 Contributions: Advancing Unsupervised Entity Resolution

This thesis and its accompanying source code introduce several novel concepts and methodologies that contribute to advancing the state-of-the-art in unsupervised entity resolution. Our key contributions towards a generic pipeline are as follows:

- **Weak Labels for Entity Resolution:** We introduce a four-step process that generates weak labels entirely unsupervised from a given set of data sources, producing a small set of training labels optimized for fine-tuning embedding language models. Additionally, we demonstrate that the weak labels alone perform remarkably well for entity resolution, particularly given their fully unsupervised nature. Specifically, our stages cover:
  - **Feature Creation Techniques:** A comparative analysis of similarity measures across various scenarios to identify those that serve as robust and generic weak label creation techniques.
  - **Threshold Selection Methods:** Three novel strategies for converting continuous similarity measures into binary labels, along with a comprehensive comparison of their effectiveness.
  - **Aggregation Algorithms:** Enhancements to existing aggregation algorithms tailored to the entity resolution challenge, resulting in a reliable yet straightforward approach.
  - **Sampling Strategies:** Application of concepts from existing sampling techniques to select training data for fine-tuning embedding models, addressing both individual instances and groups of records.
- **Optimizing Embedding Models for ER:** We investigate multiple objective functions for fine-tuning, tailored to enhance the matching performance of embedding models for entity resolution. Their effectiveness is evaluated to determine the most suitable approaches in the context of our pipeline.
- **Automating the Matching Stage:** To ensure our pipeline is fully unsupervised, we explain how a model fine-tuned with our approach can simplify hyper-parameter selection during the matching stage.

- **Experiments & Evaluation:** We conduct a variety of experiments to justify our choice of methods and parameters. Our final evaluation confirms that fine-tuning has a positive impact on the matching performance of embedding language models.

All contributions are implemented in Python and can be utilized either as standalone modules to support further research – or as an integrated pipeline, forming the foundation for a fully unsupervised entity resolution framework. The source code is publicly available for download[2].

2 https://dl.fsold.de/master-thesis-fs2025/unsupervised-er-fine-tuning-sourcecode.zip

# Part I

## Foundations & Context

# 2 Entity Resolution: Foundations, Notation & Related Works

With decades of research around *Entity Resolution (ER)*, it might come as a surprise that the field is still studied extensively. While many approaches and commercial solutions exist, they often depend on specific domains, scenarios, or domain knowledge to be available. Thus, it remains challenging for duplicate entries to be reliably detected, particularly in scenarios where matching experts are unavailable. Subsequent tasks, such as linking between entries across systems or combine multiple records into one, are heavily dependent upon the quality of the ER process.

In this section, we first introduce *Entity Resolution* formally and present key advancements in the research field. Afterward, we dive deeper into the field of unsupervised ER and discuss the core related work in this area.

## 2.1 An Introduction to Entity Resolution

The term *Entity Resolution* refers to the process of resolving "real-world" entities across various data sources, where a single entity may be represented by multiple records [Chr12]. Other terms for this challenge include *Data Matching*, *Record Linkage* or simply *Deduplication* and *Duplicate Detection*. We distinguish primarily between two settings: multi and single source entity resolution. Inherent to single source ER is that the underlying data is dirty, i.e., multiple records referring to the same real-world entity exist. As a result, this task is commonly referred to as *Dirty ER*. For multi source ER, the task can either entail to match between individually clean data sources or allow these to be individually dirty, too. The former case for two data sources has been a frequent subject to research and is commonly referred to as *clean-clean ER*. In the context of this thesis, we focus our efforts on *clean-clean ER* where each record from data source A can be matched with at most one record from source B.

The task of ER would be trivial to solve if records referring to the same real-world entity were exact duplicates (i.e., equal in all attributes). In practice, duplicate records are usually challenging to detect as they bear various errors or differences, e.g., because of typos, missing values or simply incorrect values [NH10]. Such duplicates are commonly referred to as "fuzzy" duplicates, highlighting their inequality.

| | Positive | | Negative | |
|---|---|---|---|---|
| Predicted Positive | $E \cap G$ | $(TP)$ | $E \setminus G$ | $(FP)$ |
| Predicted Negative | $G \setminus E$ | $(FN)$ | $(S_A \times S_B) \setminus (E \cup G)$ | $(TN)$ |

**Figure 2.1:** *Confusion Matrix* used to compare an experiment $E$ against a ground truth annotation $G$ on data sources $S_A$ and $S_B$. *Abbreviations: True Positives (TP), False Positives (FP), False Negatives (FN), True Negatives (TN).*

Across multiple data sources, varying schemata present an additional challenge, which is usually addressed in a separate *Schema Matching* process [RB01].

Formally, we define a data source $S$ with $n$ records $S = \{r_1, ..., r_n\}$. The schema of $S$ is given as a set of $m$ attributes $A = (A_1, ..., A_m)$ which each record represented as $r_i = (a_1, ..., a_m)$. We define a pair $p_i \in S_A \times S_B$ as the combination of two records $p = (r_j^A, r_k^B)$ from data sources A and B. Either through matching or a ground truth annotation, each pair receives a label: "match" (= 1) or "non-match" (= 0). As a result, matching two data sources requires an ER approach to provide a decision for all possible pair combinations, yielding a naive runtime of $O(|S_A| \cdot |S_B|)$.

To evaluate the effectiveness for research purposes, a scenario usually includes a ground truth annotation besides the data sources themselves. A ground truth annotation (or labelling) $G$ is a subset of all possible pairs $G \subseteq S_A \times S_B$ and represents all pairs that are true matches, i.e., that refer to the same real-world entity. To assess the set of pairs $E \subseteq S_A \times S_B$ that a matching approach identified as possible matches, we compare the prediction $E$ against the ground truth annotation $G$ in a *Confusion Matrix* (see Figure 2.1). Correctly identified matches are called "true positives" (TP) while erroneous matches are referred to as "false positives" (FP) and erroneous non-matches as "false negatives" (FN). The category of "true negatives" (TN) refers to pairs that neither ground truth nor matching approach label as a match, and commonly outweighs the other categories in the number of pairs contained by far.

This imbalance in the number of "true negative" pairs is intuitive, as the Cartesian product $S_A \times S_B$ generates many trivial, non-matching pair combinations. Consequently, evaluating the performance of a matching approach solely upon the percentage of correctly assigned labels (i.e., using the *accuracy* metric) provides an incomplete picture. Instead, the evaluation focuses on assessing *precision*, *recall* and their combination, the *F1 score* (see Definition 2.1). *Precision* measures the percentage of pairs labeled as match by the matching approach that are actually true matches, while *recall* quantifies the percentage of true matches identified by the approach. The *F1 score*, calculated as their harmonic mean, provides a balanced

metrics that highlights the trade-off between these two aspects of performance, offering a comprehensive view of an approach's matching effectiveness.

▶ **Definition 2.1.** With *TP*, *FP*, *FN*, and *TN* as illustrated in Figure 2.1, we define the metrics to assess matching effectiveness as:

$$Precision = \frac{TP}{TP + FP} \qquad Recall = \frac{TP}{TP + FN} \qquad F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

◀

## 2.2 Building Entity Resolution Pipelines

A typical ER framework or system consists of multiple stages. First, a blocking algorithm removes easy non-matches from the candidate set and thereby drastically decreases the number of pairs relevant for the subsequent matching stage. As the candidate set initially contains the Cartesian product of both data sources, blocking algorithms need to be very lightweight and simultaneously effective in reducing the set's size. As an example, partitioning can be applied to group records from both sources into disjoint groups (e.g., by zip code) [NH10], thereby limiting the candidate set to pair combinations within these groups. More recent approaches to blocking, such as Sparkly by Paulsen et al. [PGD23], use textual similarity measures like TF/IDF to identify records with overlapping content.

Next, the candidate set is thoroughly analyzed for true duplicates with a matching approach. Traditionally, these approaches were based on complex rule sets compiled by domain experts. Similarity measures, indicators for the similarity of two records, serve as the foundation of a rule set. Commonly used similarity measures include the Jaccard or Levenshtein distance. Together with a threshold, each measure is part of one or multiple rules used to determine whether a pair represents a match or not. [Chr12]

Besides rule-based matching approaches, machine learning-based approaches initially also used similarity measures as input features. With recent advancements in deep neural networks and pre-trained large language models, both also saw wide adoption in entity resolution, utilizing textual input directly instead of similarity measures as an intermediate representation. In their synthesis lecture, Papadakis et al. [Pap+21] categorize the evolution of entity resolution and its associated matching systems into four distinct generations.

Regardless of the matching approach employed, evaluating matching performance is a crucial yet complex task required to assess effectiveness and refine results.

To facilitate this process, Graf et al. built Frost [Gra+22], a framework to benchmark and explore entity resolution result sets. Besides traditional performance metrics, Frost captures *soft key-performance-indicators (KPIs)* as an additional benchmarking dimension to evaluate the economic viability of a matching approach.

## 2.3 Matching with Domain Knowledge

In practice, most matchers require domain knowledge to be available in one way or another. For rule-based approaches, the knowledge is incorporated in the similarity measures, thresholds, and rules compiled by human experts. For machine learning models, labeled training data provides the necessary context to distinguish matches from non-matches. Such matching models are commonly referred to as supervised approaches to highlight the supervision necessary to train them [BG21]. Depending on the domains used for training, such matching models are typically heavily scenario-dependent and cannot easily be used to match unseen domains.

Over the past decades, various matching systems have been developed. The *Magellan* framework [Kon+16] offers a comprehensive workflow that guides users through the core stages: deriving features from the data sources, applying a blocking algorithm and performing the actual matching stage. However, while Magellan can support the user with recommendations and convenient tooling, it ultimately relies on domain experts (or a ground truth annotation) to assess which blocking and matching rules provide sufficient results. To overcome this limitation, Efthymiou et al. introduced *PyJedAI* [Eft+23], a Python-based library designed as an end-to-end entity resolution pipeline. PyJedAI integrates various ER matchers and aims to include automated configuration, simplifying the optimization process. At the time of writing, their framework remains under active development.

Besides frameworks, both research and industry have developed numerous matching approaches based on various technologies. The latest addition to the field and the current state-of-the-art are matchers based on pre-trained language models (LMs). Such approaches benefit from the generic language understanding encoded in the model and thereby limit the need for labeled training data to fewer samples – or even circumvent training data entirely. For example, the *DeepER* [Ebr+18] matcher by Ebraheem et al. uses word embeddings like *word2vec* [Mik+13] to decrease the amount of training samples required as well as the dependency on a specific dataset. Another example is *Ditto* developed by Li et al. [Li+20] which fine-tunes pre-trained LMs, such as BERT [Dev+19], on the deduplication task and improves significantly upon prior state-of-the-art tools requiring only half the

number of training samples for many datasets. Nevertheless, Ditto still requires fine-tuning and thereby a set of labeled training samples.

In contrast, only a few task-specific samples or none are required when mapping the task to a prompt for a generic pre-trained large language model (LLM). A broader study by Peeters and Bizer [PSB25] compares the reasoning capabilities of several hosted and open-source large language models regarding entity matching effectiveness. They conclude that LLMs are a decent solution in scenarios where there is no training data available and that state-of-the-art models can provide compelling explanations for their matching decisions. Nevertheless, comparing all records solely with a language model requires up to $|S_A| * |S_B|$ queries, which, given that such models are either pay-per-use or necessitate significant computational resources, results in a significant expense. Hence, Li et al. [Li+24] propose to employ OpenAI's GPT-4 [Ope24] with a zero-shot learning approach to optimize a probabilistic model seeded with a traditional matcher. Given a certain budget, they select an optimal set of questions to pose towards the LLM.

Another group of matchers, designed to reduce the number of training samples required, are those based on *Active Learning*. In an *Active Learning* setting, training starts with a very small set of labeled training samples and is iteratively expanded through human annotation of the most uncertain instances. Identifying the most uncertain pairs requires a scoring measure derived from the model's current training state, such as the informativeness measure proposed by Christen et al. [CCR20]. However, the number of human annotations required remains scenario-dependent, making this approach a potentially time-consuming task.

## 2.4 Unsupervised Entity Resolution

In contrast, unsupervised entity resolution has received relatively limited attention in recent years. Traditionally, unsupervised approaches rely on clustering technique to bring records referring to the same real-world entity closer to one another, forming a coherent cluster. In 2006, Bhattacharya and Getoor [BG06] proposed an unsupervised matcher built on a generative Latent Dirichlet model. Their approach was specifically geared towards resolving author entities in citation datasets and benefited from assigning authors to collaboration groups. More recently, in 2021, Xu et al. [Xu+21] proposed an unsupervised approach based on random forests. Here, the authors first map each record to a semantic vector representation, then apply a random forest to partition these into entity groups and finally apply a clustering algorithm to yield the output entities. While their matcher outperforms

other clustering algorithms, its *F1 score* of 0.42 on the *Amazon-GoogleProducts* dataset leaves room for future research.

Wu et al. introduced ZeroER [Wu+20], a generic, entirely unsupervised matcher evaluated across various datasets, and against multiple unsupervised as well as state-of-the-art supervised matchers (e.g., Ditto [Li+20]). Their approach aims to distinguish similarity vectors (i.e., feature vectors derived with similarity measures) of matching pairs from those of non-matching pairs, leveraging the assumption that their distributions are significantly different. In their evaluation, Wu et al. claim that ZeroER delivers a performance competitive with active learning or supervised approaches. However, as Zeakis et al. [Zea+23a] demonstrate (refer to Section 4.1, Figure 4.2), ZeroER's matching performance fluctuates significantly when tested on a broader range of scenarios and does not always reliably terminate. In our own experiments, we observed that ZeroER heavily relies upon an optimal and computationally efficient blocking function, which must be defined by a proficient user. This dependency ultimately limits ZeroER's claim of being fully unsupervised and demonstrates the limitations of unsupervised entity resolution.

Luckily, language models enable an entirely new generation of unsupervised entity resolution approaches. However, unlike traditional unsupervised approaches such as ZeroER, LMs are pre-trained – either for general-purpose tasks across diverse domains or tailored to a specific task or use-case. While this pre-training supports generality, it often limits matching effectiveness for less generic scenarios. Zeakis et al. [Zea+23a] evaluated the performance of pre-trained embedding language models across ten scenarios, providing a conceptual foundation for the pipeline we propose. Our approach aims to build upon the promising performance of LMs with scenario-specific fine-tuning – relying solely on automatically generated weak labels. Compared to both ZeroER and pre-trained embedding models, our pipeline delivers more robust and consistent results with an entirely unsupervised, automated workflow.

# 3 Language Modeling: Core Concepts & Applications

In the earlier chapters of this thesis, we frequently referenced the core idea of fine-tuning embedding language models with weak labels. To provide the necessary context, we dedicate the following chapter to the foundational concepts from the area of language modeling upon which our pipeline is built.

The first section of this chapter explores the core concepts of language models, with a focus on the design and functionality of embedding models. Here, we also explain the architecture of the underlying language model family and how the *st5-base* embedding model we utilize was created. Finally, we introduce the principles of weak supervision and weak labels, a novel strategy for training machine learning models effectively using noisy or unreliable labels.

## 3.1 Natural Language Modeling & Embeddings

Scientists have been modeling language for more than 100 years. In fact, one of the earliest applications of the Markov chain from probabilistic theory was the modeling of language. Nearly 90 years later, the first neural language model was proposed, which also pioneered the concept of word embedding vectors [Li22]. In contrast to earlier approaches that relied on large vocabularies and simply encoded the existence of a word, embedding vectors have a fixed dimensionality and are designed to capture the semantic meaning of a word. In 2013, engineers at Google AI[3] developed Word2Vec [Mik+13], a family of "shallow" models that can be used to efficiently learn word embeddings within linguistic contexts from large datasets. As an additional note, a pre-trained model from this family, trained on the *Google News 300* dataset, will be used as a semantic similarity measure in Subsection 5.1.4.

Since then, more complex model architectures and particularly the *Transformer Architecture* based on the self-attention mechanism [Vas+17] enabled entirely new model families. The core concept of "self-attention" is the ability to contextualize a word (or a token as a part of a word) with the entire input sequence, while simultaneously lowering the computational complexity compared to traditional recurrent or convolutional network architectures. One of the most popular models built with transformers is BERT by Devlin et al. [Dev+19] at Google AI. BERT is

3 https://ai.google

trained on two key tasks: masked token and next sentence prediction. Through this process, it simultaneously learns contextualized embedding vectors as intermediate representations of input sequences, which can also be utilized independently. In the past 5 years, language models grew even "larger" and further improved their performance in various aspects, culminating in models such as OpenAI's GPT-4 [Ope24] or Meta's Llama[4] model family.

Particularly relevant to this thesis are derivative works based on pre-trained language models, such as the work of Reimers and Gurevych [RG19]. They propose a modified version of BERT called *SentenceBERT* trained for *Semantic Textual Similarity (STS)* using a siamese network structure. The *Semantic Textual Similarity (STS)* task is about determining how similar two pieces of text (or in our case two records) are. In other words, sentences (as sequences of tokens) are mapped to embedding vectors such that vectors of semantically similar sentences entail a high similarity.

Zeakis et al. [Zea+23a] demonstrated that the STS task is similar enough to entity resolution such that pre-trained embedding models already provide decent matching performance in some scenarios. Similarly to the authors, we focus on the pre-trained embedding model family *st5* for our pipeline which is based on the *T5* text-to-text transformer model. The *T5* model family was developed by Raffel et al. [Raf+20] in 2020 and features a fully-connected encoder-decoder architecture. In such an encoder-decoder architecture, the tasks of interpreting and generating text are strictly split between encoder and decoder. The model is pre-trained for various text-to-text tasks, such as translation or summarization.

To build upon this profound language understanding, Ni et al. [Ni+22] studied how text-to-text transformers by example of *T5* can be transformed into sentence embedding models. They conclude that *T5*'s encoder, combined with mean pooling across all individual token embeddings, outperforms *SentenceBERT* significantly. From a structural perspective, their *sentence-t5* models utilize a siamese dual-encoder network (see Figure 3.1). The core idea thereby is to use the pre-trained model's encoder twice with shared parameters (i.e., the model's weights) and score the embeddings they generate with a similarity function. To train the model for the STS task, the achieved similarity score can be compared to a target value, and the loss back-propagated simultaneously.

In the context of this thesis, the ability to fine-tune pre-trained *sentence-t5* models for a specific target domain or use-case is of particular importance. While these pre-trained models already demonstrate strong performance across various domains, Zeakis et al. [Zea+23a] highlight that such performance cannot be reliably assumed

4 https://llama.com

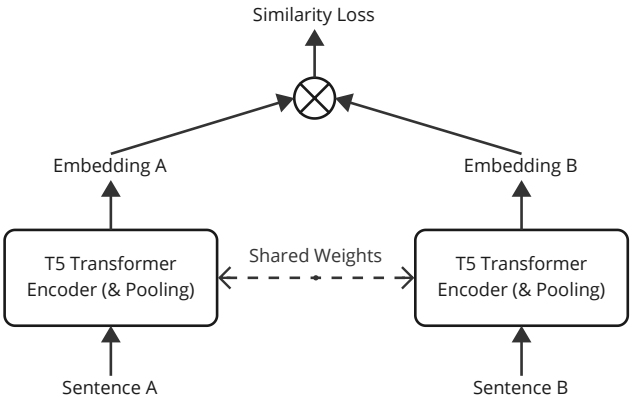


**Figure 3.1:** Siamese dual-encoder network structure used to pre-train the *sentence-t5* model family for the STS task (simplified architecture).

for entity resolution. This is because a record's textual context often lacks the coherence of a natural sentence, which language models are optimized to process. This assumption further underscores the opportunity to enhance robustness in entity resolution tasks through fine-tuning. The expected impact of fine-tuning is twofold: first, optimizing the model for processing scenario-specific records, and second, improving its ability to effectively distinguish between matches and non-matches.

## 3.2 Empowering Models with Weak Supervision

The final piece of the puzzle for this thesis is the concept of weak labels, which our pipeline uses to fine-tune pre-trained embedding language models for scenario-specific ER. Traditionally, supervised model training relies upon reliably annotated datasets. During training, these label annotations act as the target behavior, guiding the optimization of the model's weights. However, as outlined in Chapter 1, obtaining such high-quality labels is often expensive and time-consuming, posing a significant barrier to practical applicability.

A common approach to labeling new datasets is through crowdsourcing, where a global workforce manually annotates each sample. However, humans are not always consistent or reliable in the labels they assign, especially when working with complex or ambiguous datasets. To address this issue, it is common practice to aggregate the labels from multiple annotators into a single, more reliable and

less noisy label. Similarly, our pipeline incorporates label aggregation, a process we discuss in detail in Section 5.3.

Instead of relying on human annotators, *Programmatic Weak Supervision (PWS)* leverages noisy *labeling functions (LFs)* to generate training labels directly from the dataset [Zha+22]. Once a set of labeling functions is developed, generating multiple "noisy" labels for a given sample incurs no additional cost, allowing for the efficient labeling of large datasets. Related methodologies, such as *Semi-Supervised Learning* and *Distant Supervision*, similarly aim to reduce dependence on fully annotated datasets.

In contrast to PWS, *Semi-Supervised Learning* combines a small set of human-annotated samples with a larger unlabeled or poorly labeled dataset. The core idea is to benefit from patterns and structures within the entire dataset alongside the labeled subset to improve model performance. *Distant Supervision*, on the other hand, requires no human annotation and operates similar to PWS. It relies on an external knowledge source (e.g., Wikipedia[5]) to automatically extract labels and annotate the dataset. However, this process is inherently noisy, as its accuracy depends on the reliability of both the knowledge source as well as the extraction process.

*Programmatic Weak Supervision (PWS)* addresses this challenge by combining multiple labeling functions with varying accuracy into a single, more reliable label. Snorkel, introduced by Ratner et al. [Rat+17], is one of the earliest and most influential approaches in PWS. It provides an end-to-end framework for defining labeling functions and subsequently applying unsupervised models to aggregate their outputs into a single label per sample (see Subsection 5.3.4). In summary, PWS reduces the labeling effort while tying label quality to the effectiveness of the LFs used, making it a practical solution for scenarios where reliable annotations are scarce.

For several decades, similarity measures have been the cornerstone of entity resolution systems in both research and industry [Chr12]. This makes them ideal candidates to build labeling functions with. At a higher level, rule-based matching approaches can be transformed into a PWS framework: individual predicates (i.e., combination of similarity value and respective threshold) serve as labeling functions, while the entire rule set is similar to the aggregation of LFs in PWS.

While this approach appears straightforward at first glance, the main primary challenge lies in constructing an entirely automated, unsupervised entity resolution pipeline around it. Without this aspect of automation, users would still face a

5 https://wikipedia.org

significant effort akin to creating rule sets manually, severely limiting the pipeline's practicality and scalability.

To address this challenge, our pipeline utilizes the concept of weak labels, generated through PWS, to fine-tune the embedding language model *st5-base* for scenario-specific entity resolution. In the subsequent Chapter 4, we introduce our pipeline and describe its entirely unsupervised, automated weak label creation process.

# Part II

# The Entity Resolution (ER) Pipeline

# 4 Applying the Technology: Embeddings for Unsupervised ER

The central idea of this thesis is to combine (embedding) language models and traditional similarity metrics as complementary knowledge sources for autonomous, unsupervised entity resolution. While the database community has proposed numerous approaches to use language models for the entity resolution task (see Section 2.2), only a limited number of publications address the unsupervised setting. This gap is largely due to the prevailing focus on training or fine-tuning language (or generic deep learning) models using a ground truth annotation.

For unsupervised entity resolution, Zeakis et al. published a detailed analysis comparing multiple embedding language models for various entity resolution tasks (abbreviated as *Embeddings4ER* or *E4ER*) – specifically also for blocking and the unsupervised setting [Zea+23a]. Besides the short version presented at VLDB 2023, a longer version of this work with additional evaluations has been published on arXiv [Zea+23b] and the respective source code, as well as result data, are provided on GitHub[6]. Our goal is to build upon the already significant, unsupervised matching performance of language models as evaluated by Zeakis et al. with weak-label fine-tuning. Consequently, our methodology closely adheres to their evaluation framework, allowing for a meaningful comparison of our results with their work as a baseline. The concepts behind our pipeline align well with the authors' vision of combining their findings into a "learning-free approach to ER with SentenceBERT models" [Zea+23a].

In the following sections, we first thoroughly explain how pre-trained embedding models can be used to achieve unsupervised entity resolution. Afterward, the main benchmark scenarios as proposed by Zeakis et al. are introduced, and we explain our methodology. Finally, we introduce our pipeline, which represents the core contribution of this thesis.

## 4.1 Pre-trained Embeddings for Entity Resolution

As alluded to in Section 3.1, pre-trained embedding language models bear a semantic understanding of natural language that is capable of identifying semantically similar sentences (or words). This language model task is commonly referred to as semantic

6 https://github.com/alexZeakis/Embeddings4ER

textual similarity (STS), and Zeakis et al. demonstrated that the entity resolution task is indeed similar enough to STS to achieve decent performance in the unsupervised setting. For this to work, they map each record to the concatenation of all its attributes, such that it is represented as a single "sentence".

On a methodology level, the authors define three core tasks that were thoroughly analyzed for their effectiveness. All of them assume the existence of a vector representation (or embedding) for each record within the data sources.

- **Blocking:** Extract a candidate set of pairs from all possible pair combinations that is as small as possible, but still contains as many real duplicate pairs as possible. The effectiveness is assessed with the recall measure across different candidate set sizes.

- **Unsupervised ER:** Given a similarity score for each pair of embedding vectors, provide a matching such that all records referring to the same real-world entity are grouped into the same "cluster". As their matching approach *Unique Mapping Clustering* (UMC) requires a threshold hyper-parameter to be set, they compare using the best *F1 score* across a set of threshold values.

- **Supervised ER:** Based upon the embedding vectors, train a downstream neural network on a fixed set of ground-truth training samples to distinguish between matches and non-matches. The effectiveness during training (and testing) is measured using the *F1 score*.

In this thesis, our primary focus is on evaluating the blocking and unsupervised entity resolution performance of embedding models. Both tasks were evaluated in a clean-clean setting where two individually duplicate-free data sources are compared to identify matching records. According to the referenced study, the model with overall best performance in blocking and unsupervised ER is "S-GTR-T5 (base)" (*st5-base*). While its runtime is relatively high compared to other embedding models, this limitation is negligible for our use case. Consequently, all subsequent experiments are focused on and conducted using this model.

Contrary to common practice, the authors do not apply blocking as part of the unsupervised entity resolution task. Instead, they search for a matching upon all possible pair combinations between the two data sources and justify this practice with a possible negative influence of blocking on their matching technique *Unique Mapping Clustering*. In contrast, our pipeline leverages the power of blocking to identify a promising subset of candidate labels and thereby limit the costly generation of weak labels.

**Vectorization.** An implicit preliminary step for both blocking and matching involves mapping each record to a text sequence that the embedding model vectorizes. For this purpose, the authors aggregate all attribute values of each record into a single string, separated by whitespace (see Section 4.2). The resulting embeddings generated by the model serve as the foundation for both blocking and unsupervised entity resolution.

**Blocking.** As part of their blocking experiments, Zeakis et al. carry out an exact *nearest neighbor search (NNS)* to identify, given a record in dataset source A, the $k$ most similar records in source B. The provided source code reveals that they leveraged the Euclidean distance as the comparison measure and therefore select the $k$ vectors with the smallest distance to the target vector as candidates. In their paper, they also mention the possibility of using an approximate HNSW[7] index as provided by the FAISS[8] library for cases in which NNS becomes too computationally expensive to perform – solely for the larger data sources used in the supervised ER experiments.

As part of their experiments, the authors demonstrate that *st5-base* outperforms the state-of-the-art (SotA) approach *DeepBlocker* [Thi+21] in all scenarios and for $k \in 1, 5, 10$. The hosted source code additionally contains baseline results of Sparkly [PGD23], which shows competitive blocking effectiveness. In alignment with the authors, we conclude that language embedding models, specifically *st5-base*, can serve as an effective blocking step in an end-to-end ER pipeline.

For our work specifically, we use a similar exact NNS implementation provided by FAISS that uses the inner product of both vectors as a similarity measure. Given two normalized vectors, their inner product is equivalent to their cosine similarity. Furthermore, we set $k = 5$ as a tradeoff between high recall and a small candidate set contrary to $k = 10$ in their paper. To justify this choice over the implementation provided by Zeakis et al., we ran comparative evaluations on blocking recall (see Figure 4.1). As one can see, our approach with $k = 5$ remains competitive to their implementation even at its highest value of $k = 10$ and provides more consistent performance compared to Sparkly.

**Unsupervised Entity Resolution.** As Zeakis et al. model unsupervised entity resolution as a bipartite graph matching, multiple algorithms exist that can effectively solve this task. On an algorithmic level, records are represented as nodes and connected with edges weighted by their similarity measure. Again, the authors make use of the Euclidean distance as a similarity measure and compute similarities for the entire set of possible pairs. As for the matching algorithm itself, prior works

7 Hierarchical Navigable Small Worlds (HNSW)
8 FAISS is a Python library for efficient vector similarity search. https://faiss.ai

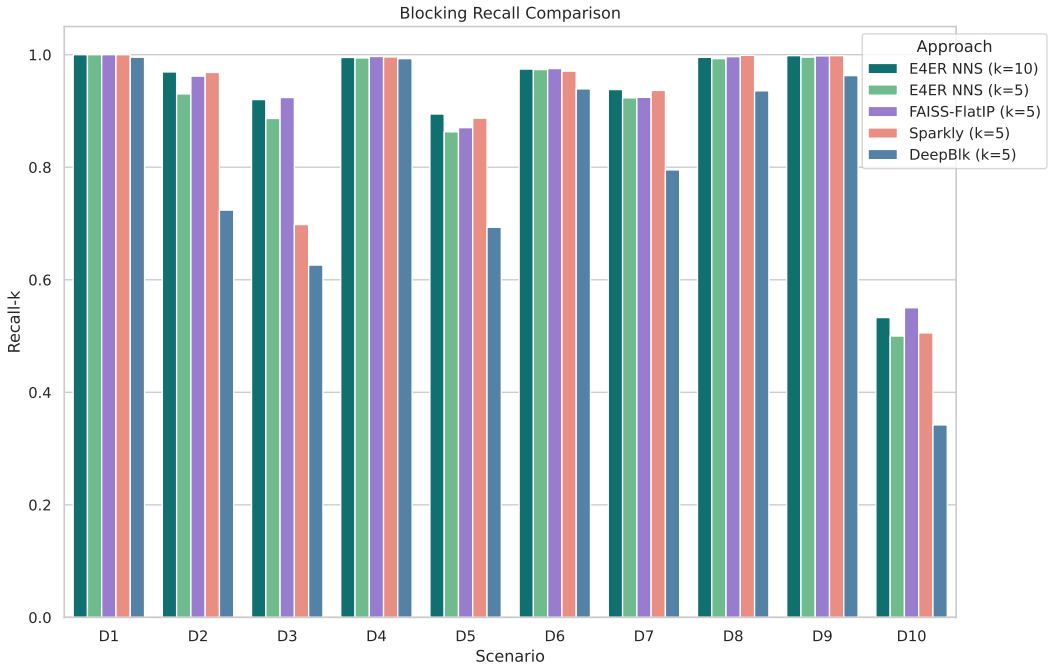


**Figure 4.1:** Blocking performance comparison of Embeddings4ER (E4ER NNS), Sparkly, DeepBlocker (DeepBlk) and our approach based on FAISS-FlatIP for $k = 5$ (and additionally $k = 10$ in the case of E4ER NNS). Results are partly sourced from Zeakis et al. [Zea+23b].

by Papadakis et al. [Pap+22] empirically evaluated various bipartite graph matching algorithms specifically for the clean-clean entity resolution task. In alignment with their conclusion, Zeakis et al. chose *Unique Mapping Clustering (UMC)* "which achieves both high effectiveness and time efficiency" [Zea+23a].

One particular downside of this approach is the dependency upon a hyper-parameter, $\delta$, that represents the threshold value below which all edges are pruned from the graph. Quite intuitively, $\delta$ exhibits a significant influence on the matching effectiveness and ultimately the overall performance. Choosing the optimal value $\delta$ becomes crucial, as the *F1 score* may drop as low as 0.0 if chosen inadequately (refer to Section 6.4 for further details). As stated in the paper, the authors are aware of this dependency and intentionally chose not to discuss possible threshold selection techniques. Instead, they select the best possible *F1 score* among a set of predefined $\delta \in [0.05, 0.95]$ with a step size of 0.05, rendering their approach not entirely unsupervised.

Zeakis et al. identify ZeroER [Wu+20] as the primary competing approach. Figure 4.2 compares the performance of the best embedding model selected by the authors, *st5-base* against both ZeroER and the model's larger sibling *st5-large*. As one can observe, *st5-base* provides consistent performance and is competitive with ZeroER, which was only able to terminate for half of the scenarios in a reasonable timeframe. Contrary to what was stated in the paper, the larger *st5-large* model does perform noticeably better in all but one scenario. Consequently, we compare our results both against *st5-base* and *st5-large*, while we solely fine-tune *st5-base* with weak labels to retain comparability. Evaluating more powerful embeddings models, such as *st5-large*, to analyze their strengths, weaknesses and optimal fine-tuning process, constitutes a distinct research project in its own.

**Fine-tuning.** Our pipeline focuses on fine-tuning embedding models with weak labels and hence assumes that domain-specific samples improve the model's understanding – and consequently, its ability to reliably identify true duplicates. At first glance, this assumption contrasts with the findings of Zeakis et al. who observe only a minor benefit from fine-tuning embedding models. The authors base their assessment upon their experiments training embedding models for supervised entity resolution. Furthermore, they conclude that this behavior may be caused by a lack of coherence within the "sentence" as it is formed by aggregating all attribute values. As we demonstrate in the following sections, fine-tuning with weak labels can significantly benefit an embedding model's matching capabilities.

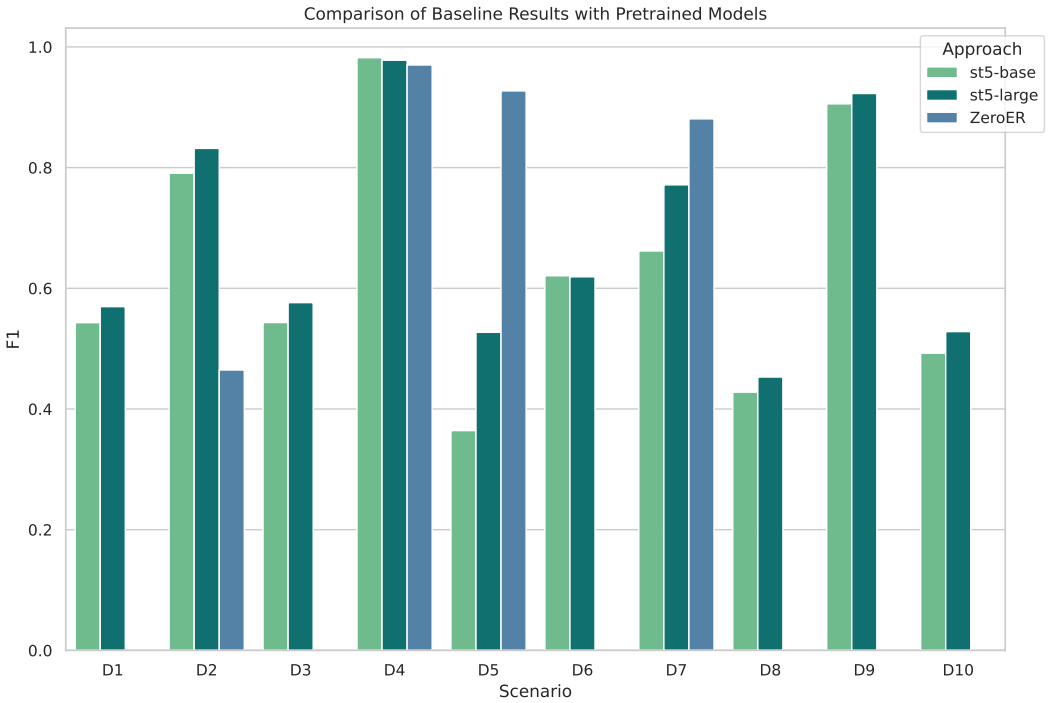


**Figure 4.2:** Comparison of baseline results for the language models *st5-base* and *st5-large* against the state-of-the-art approach ZeroER across all ten scenarios provided by Zeakis et al. Results for ZeroER, where execution completed, are sourced from Zeakis et al. [Zea+23b].

## 4.2 Development & Evaluation Scenarios

To evaluate blocking and unsupervised entity resolution, Zeakis et al. [Zea+23a] make use of ten matching scenarios for clean-clean ER that have been subject to extensive prior research. The data sources used within each scenario vary in size ($|D_1|$ and $|D_2|$), form (e.g., count of attributes $|A_1|$ and $|A_2|$), count of true matches ($|M|$) and target domain, but predominantly appear to be easy to match (see Table 4.1). Furthermore, we need to assume that pre-trained (embedding) language models may have had access to some of these sources during their training.

The authors enriched the data sources by providing an *aggregate value* for each record, more specifically, crafted by aggregating all attribute values of each record into a single string, separated by whitespace (*schema-agnostic* representation). While this approach is particularly straightforward, it entirely neglects positional information provided through column names. Other entity resolution frameworks include this information, such as Ditto by Li et al. [Li+20], who utilize the format "COL <name> VAL <text> COL ..." instead (*schema-based* representation). To preserve comparability against the baseline results, our experiments also utilize the aggregate value provided by Zeakis et al.

Below, we introduce the ten scenarios and describe their characteristics, particularly the data sources they are based upon.

- **D1.** This small scenario[9] features restaurant listings from two data sources where each restaurant is identified by name, address, style, and phone number.
- **D2, D3 & D8.** These scenarios contain product information extracted from Abt & Buy, Amazon & Google [KTR10] and Walmart & Amazon [Mud+18]. All three data sources feature around a thousand true duplicates and are primarily matched on text-heavy attributes, such as product names and descriptions, present in both sources.
- **D4 & D9.** Both D4 & D9 [KTR10] match citations from DBLP with ACM or Google Scholar, respectively. Besides shorter attributes such as year or venue, records also contain the full paper name and the list of authors.
- **D5, D6 & D7.** For these three data sources [OSR21], all pairs of IMDb, TMDb and TVDB are to be matched. While the topic of movies is the same for all sources, they do significantly differ in the attributes they provide. For instance, IMDb offers generic information about movies, whereas TVDB is

9 http://oaei.ontologymatching.org/2010/im

| Scenario | $\mathbf{D_1}$ | $\mathbf{D_2}$ | $\mathbf{D_3}$ | $\mathbf{D_4}$ | $\mathbf{D_5}$ | $\mathbf{D_6}$ | $\mathbf{D_7}$ | $\mathbf{D_8}$ | $\mathbf{D_9}$ | $\mathbf{D_{10}}$ |
|---|---|---|---|---|---|---|---|---|---|---|
| $\mathbf{Source_A}$<br>$\mathbf{Source_B}$ | $\text{Rest}_1$<br>$\text{Rest}_2$ | Abt<br>Buy | Amz<br>GPr. | DBLP<br>ACM | $\text{IMDb}_1$<br>TMDb | $\text{IMDb}_1$<br>TVDB | TMDb<br>TVDB | Wmt<br>Amz | DBLP<br>Scholar | $\text{IMDb}_2$<br>DBP |
| $\lvert S_A \rvert$ | 339 | 1,076 | 1,354 | 2,616 | 5,118 | 5,118 | 6,056 | 2,554 | 2,516 | 27,615 |
| $\lvert S_B \rvert$ | 2,256 | 1,076 | 3,039 | 2,294 | 6,056 | 7,810 | 7,810 | 22,074 | 61,353 | 23,182 |
| $\lvert A_A \rvert$ | 7 | 3 | 4 | 4 | 13 | 13 | 30 | 6 | 4 | 4 |
| $\lvert A_B \rvert$ | 7 | 3 | 4 | 4 | 30 | 9 | 9 | 6 | 4 | 7 |
| $\lvert M \rvert$ | 89 | 1,076 | 1,104 | 2,224 | 1,968 | 1,072 | 1,095 | 853 | 2,308 | 22,863 |
| $F1(\textit{st5-base})$ | 0.543 | 0.791 | 0.543 | 0.982 | 0.364 | 0.621 | 0.662 | 0.428 | 0.906 | 0.492 |
| $F1(\textit{st5-large})$ | 0.570 | 0.832 | 0.576 | 0.978 | 0.527 | 0.619 | 0.771 | 0.453 | 0.923 | 0.528 |
| $F1(\textit{ZeroER})$ | 0.0 | 0.465 | – | 0.970 | 0.927 | – | 0.881 | – | – | – |

**Table 4.1:** Scenarios for clean-clean ER as provided and used by Zeakis et al. to evaluate blocking and unsupervised matching effectiveness. *F1 scores* of pre-trained embedding models *st5-base*, *st5-large* and state-of-the-art approach ZeroER [Wu+20] are also included, analogous to Figure 4.2.

focused on person-movie relations. As a result, the overlap in attributes among the three sources is minimal.

- **D10.** Similar to the previous movie scenarios, D10 [Pap+11] compares the two movie sources, IMDb and DBP, where the sole common attribute is "title". However, its IMDb records do not overlap with those seen prior in D5 or D6.

Most of the above-mentioned matching scenarios were proposed more than a decade ago and have been used to create and evaluate numerous entity resolution approaches. Consequently, they have already been thoroughly studied for their characteristics and difficulty, for example in a detailed analysis by Primpeli and Bizer [PB20] or Papadakis et al. [Pap+24].

Primpeli and Bizer proposed a set of profiling dimensions including schema complexity, textuality, sparsity, and corner cases. Based on these dimensions, scenarios can be assigned to one of five groups. *D4, D8 & D9* were assigned to group "Textual Data, Few Corner Cases" and the authors conclude that "the challenge imposed by the textuality dimension becomes trivial in the absence of corner cases" [PB20]. Furthermore, we add that embedding models benefit from high textuality, further lowering the difficulty of these scenarios. *D2 & D3* were assigned to group "Textual Data, Many Corner Cases" which, in contrast, suggests a higher overall matching difficulty. The authors state that "embedding-based matching methods have shown to achieve better results than methods relying on symbolic features [on these tasks]" [PB20].

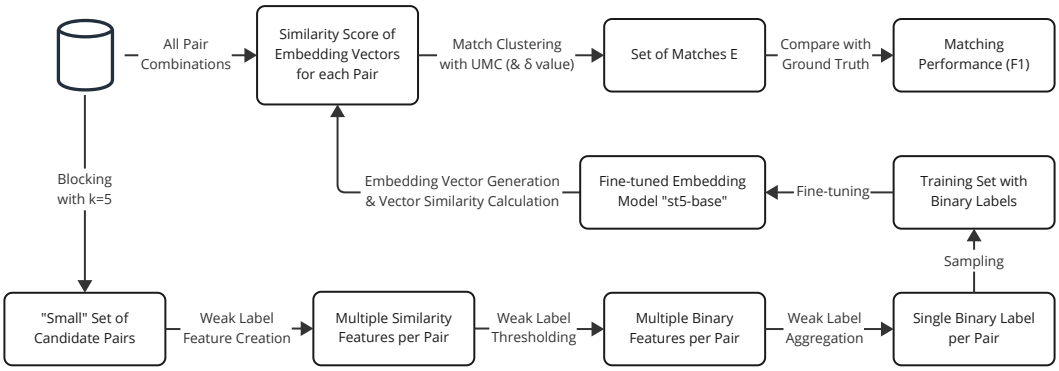


**Figure 4.3:** Flow chart of our pipeline, combining a matching stage as proposed by Zeakis et al. with fine-tuning based on weak labels. Binary label creation for weak labels is carried out entirely unsupervised using traditional similarity metrics.

A recent study by Papadakis et al. provides insights on the same subset of scenarios, echoing similar criticisms. The authors conclude that "most existing benchmark dataset [scenarios] pose rather easy classification tasks" and "are not suitable for properly evaluating learning-based matching algorithms" [Pap+24].

Although we acknowledge the limitations of certain scenarios selected by Zeakis et al., we have decided to include them to ensure comparability with prior work. Consequently, the following sections utilize these scenarios for the development and evaluation of our pipeline.

## 4.3 Our Pipeline for Unsupervised Entity Resolution

The core idea of our pipeline is to extend an embedding-based unsupervised ER approach by fine-tuning with automatically derived weak labels. For this, we base our concept on an unsupervised ER approach that uses embedding models to match records – in this case, the evaluation approach for unsupervised ER described by Zeakis et al. [Zea+23a] with model *st5-base*. Figure 4.3 outlines our pipeline in further detail. It can be split into a training and an inference part (top row of the figure). The latter part is unchanged compared to how Zeakis et al. describe their unsupervised ER approach using embedding models – we solely exchange the embedding model used with a model fine-tuned on the same scenario. Hence, we now focus on providing a detailed overview of the training part of

our pipeline. Although our description leverages the *st5-base* model, any other compatible embedding model could be applied analogously.

- **Blocking.** As a first step, we need to limit the cross-product of possible pair combinations down to a meaningful, small candidate set. Here, we apply blocking with $k = 5$ based upon a nearest neighbor search as proposed by Zeakis et al. The embedding model used is the pre-trained version of *st5-base*. However, contrary to their implementation, we leverage a FAISS-FlatIP index to conduct the nearest neighbor(s) search. As illustrated in Figure 4.1, our blocking implementation matches or outperforms theirs for all scenarios.

- **Weak Label Feature Creation.** For those pairs that remain after blocking, we then create weak label features (see Figure 4.4). More precisely, all common attributes are mapped, depending on their data type, to one or multiple similarity features. Techniques used include character-based, token-based and semantic similarity measures (among others). Additionally, there exist measures, such as TF-IDF with cosine similarity, which compare the aggregated value of all attributes and thereby account for differing schemas. At the end of this operation, we yield a vector for each pair with one or more similarity values for each attribute that illustrates how similar the two records are along multiple axes.

- **Weak Label Thresholding.** Based upon the continuous similarity vector, we approximate a threshold for each similarity measure such that we can map it to a binary label. Each threshold is thereby specifically created for one similarity measure applied to a single column and optimized to differentiate matches from non-matches. Quite intuitively, the effectiveness of this differentiation is heavily dependent on the similarity measure and column combination. To make it more tangible, a text-heavy attribute combined with a character-based similarity measure provides less differentiation than the same measure applied to an attribute of names.

- **Weak Label Aggregation.** Based upon a binary similarity vector for each pair, we successively run an aggregation operation to map each pair to a single binary label. Since this label is supposed to serve as a training signal during fine-tuning, we require that the aggregation algorithm can account for similarity measure that provide less differentiation.

- **Weak Label Sampling.** Once we have a labeled candidate set, we yet have to decide which pairs to show to the embedding model during training. Ideally,

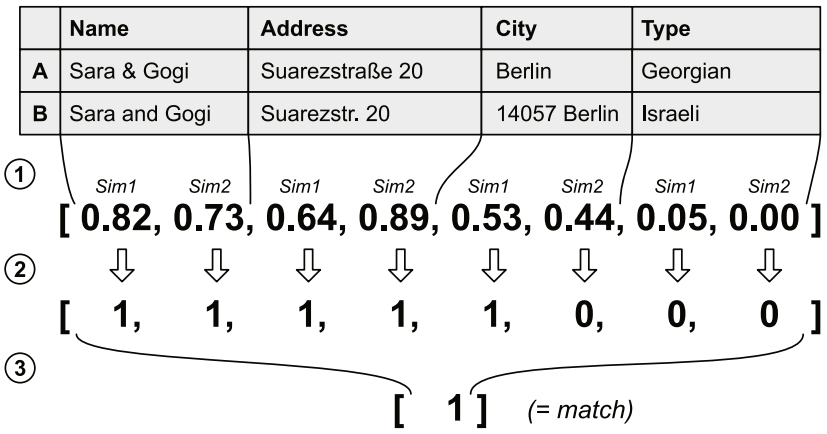


**Figure 4.4:** An example illustrating the creation of a weak label from a candidate pair consisting of records from data sources $A$ and $B$. In Step 1, two similar measures, $Sim_1$ and $Sim_2$, are applied to each attribute to generate a total of 8 similarity features. Next, in Step 2, a thresholding strategy is applied to each feature separately, converting them into binary labels. Finally, in Step 3, these binary features are aggregated into single label per pair, classifying the pair as either a match or a non-match.

> we want to show a carefully selected, confident subset. Additionally, we have the possibility to enrich individually selected pairs with supporting pairs or counter examples. Whether individual labeled pairs or groups of records are returned as training samples depends on the approach to fine-tuning that is chosen in the subsequent stage.

We apply blocking to yield a small candidate set and multiple steps to label each of the remaining pairs with a binary label. For the subsequent fine-tuning stage, we provide multiple training settings. In the most straightforward case, pairs from both classes (match and non-match) are sampled and shown to the model individually. More advanced training settings allow enriching one pair labeled as match with one (or several) counter examples as a single training instance (see Section 6.1). After a certain number of training epochs, the model is saved and used for matching on the same scenario.

As outlined above, the entire training part of our pipeline is fully automated and the weak label generation process is entirely unsupervised. Quite intuitively, this requires us to take some architectural decisions. First and foremost, this work focuses exclusively on the *clean-clean* ER setting. While *Dirty ER* could be addressed similarly, we opted to refine and optimize our approach specifically for this setting to

remain comparable with Zeakis et al. Furthermore, beyond the initial blocking stage, we impose no additional restrictions on the candidate set, potentially increasing the computational complexity for certain scenarios. As a result, some algorithms within our approach require a GPU-accelerator (refer Section 6.2 for details on our setup).

An inherent strength of our pipeline is its adaptability: each component of the weak label generation process could either be modified or replaced, allowing the pipeline to evolve alongside advancements in weak label research. During the development of this thesis, the generic entity resolution framework *pyJedAI*[10] was introduced, featuring implementations for both ER settings using embedding models. Similarly, the *RecordLinkage*[11] framework provides tools for ER tasks. Notably, our approach to weak label generation could be easily integrated into either of these frameworks.

In summary, we introduced our pipeline and explained how and where we interact with the existing concept presented by Zeakis et al. Our pipeline serves as a flexible framework that can be extended based on future research and embedded into existing matching systems. As the next step, we conduct a detailed analysis of each stage in the weak label generation process and individually evaluate the architectural decision made at each stage. Finally, we assess the overall performance of our entire pipeline against the baseline values presented in Figure 4.2.

**10** https://pyjedai.readthedocs.io
**11** https://github.com/J535D165/recordlinkage

# 5 Crafting Signals from Noise: Generating Weak Labels

The core idea of our pipeline is to optimize the matching performance of pre-trained embedding models by fine-tuning on weak labels derived from similarity measures. While embedding models, such as those from the SentenceBERT family, are already optimized for the semantic textual similarity (STS) task, they lack domain- and dataset-specific samples of true matches. Hence, we assume that we can improve this understanding by fine-tuning with a representative subset of automatically generated weak labels. The main challenges thereby lie in (I) the selection of meaningful similarity measures (II) with individual threshold, (III) an effective mapping to a single binary label, and (IV) the selection of a representative training subset. Ultimately, generating optimal weak labels is equivalent to solving the entity resolution task in the first place.

In this section, we introduce each stage around the generation of weak labels in greater detail. Besides an overview of techniques that we evaluated, we also conclude how each stage is represented in our final pipeline setup.

## 5.1 Creation: Applying Similarity Measures

Similarity measures are the core of many entity resolution approaches [Chr12]. In rule-based frameworks, these are directly applied through expert-defined thresholds and logical formulas, while machine learning models incorporate them as feature inputs, learning their meaning during the training phase. Either way, similarity measures are powerful indicators to differentiate matches from non-matches.

Throughout the past decades, various similarity measures have been proposed and thoroughly evaluated in numerous publications [DHI12]. In the context of this thesis, we differentiate between character-based, phonetic, token-based, semantic, textual and numeric measures. Additionally, we apply them either on individual attributes or on their aggregate value.

Identifying the optimal similarity measure for an attribute in a given scenario presents by itself a significant challenge. Attributes with text-heavy, diverse values benefit from token-based measures, whereas attributes with only subtle variations may be better suited with an character-based measure. Machine learning approaches address this challenge during the training phase: the model determines

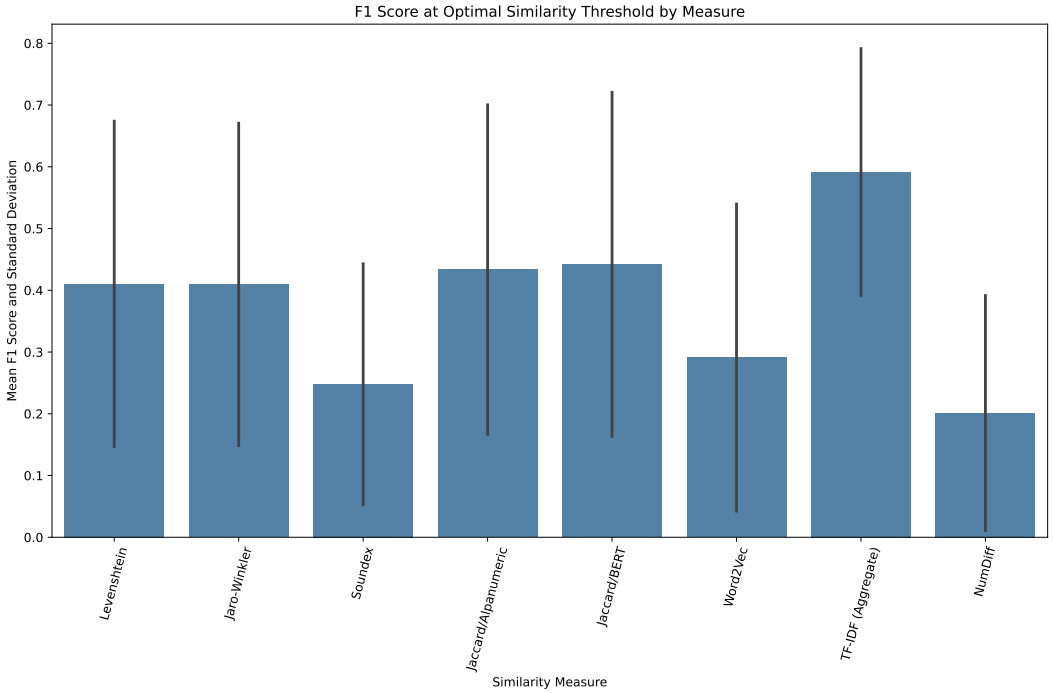


**Figure 5.1:** Performance of all similarity measures individually applied to each attribute and evaluated using the optimal threshold per attribute against the ground truth. Each measure is represented through its mean *F1 score* and standard deviation over all attributes (if applicable by data type).

by itself which features to incorporate and to which degree each should be utilized. For manual feature creation, Primpeli & Bizer [PB20] propose a differentiation into numeric, short textual, and long textual values that they use as part of their dataset profiling study. To streamline the feature creation process, our approach distinguishes only between numeric and textual values. As a result, each attribute shared by both schemas can be represented by multiple similarity features simultaneously. The aggregation of these individual similarity values is deferred to a later stage in the pipeline.

Even within a category of similarity measures, such as character- or token-based measures, there exist multiple options to choose from. In the following subsections, we individually introduce various measures and ultimately conclude on a set of measures to take forward. To evaluate their performance, we test how effectively each measure identifies true matches within the candidate set – individually for each applicable attribute. In essence, a measure contributes to the creation of a weak label in case it provides sufficient differentiation of between the ground truth

labels match and non-match. Figure 5.1 summarizes this evaluation, presenting the mean *F1 score* achieved by each measure on individual columns with an optimal similarity threshold (refer to Appendix A for further details). Ultimately, our goal is to identify a diverse set of measures that performs well as an ensemble in a wide variety of domains and is not applicable to only one specific area.

### 5.1.1 Character-based Measures

The **Levenshtein** (or edit) distance, as proposed by Wladimir Lewenstein in 1965, counts the minimal number of edit operations necessary to transform one string into another. Within the entity resolution community, the measure has been widely used and is part of popular frameworks, for example Magellan by Konda et al. [Kon+16]. Since then, other measures have been proposed that use Levensthein distance in one way or the other, such as the Monge-Elkan measure, which includes token-based scoring [DHI12]. We tested the normalized Levenshtein similarity score as provided by *py_stringmatching*[12]

William E. Winkler proposed the **Jaro-Winkler similarity** in 1990 [Win90] as a variation of the Jaro measure published by Matthews A. Jaro in the year prior [Jar89]. While the Jaro measure is based on counting matching characters and their transpositions, Winkler additionally incorporates a component that specifically rewards strings with a higher similarity that share a common prefix.

▶ **Definition 5.1.** Given two strings $s_A$ and $s_B$ with $m$ matching characters and $t$ as the number of transpositions, we define their Jaro similarity as:

$$jaroSim(s_A, s_B) = \begin{cases} 0 & \text{if } m = 0, \\ \frac{1}{3}\left(\frac{m}{|s_A|} + \frac{m}{|s_B|} + \frac{m-t}{m}\right) & \text{otherwise.} \end{cases}$$

With the length $l$ of the common prefix of $s_A$ and $s_B$ and $p \leq 0.25$ as a constant scaling vector, we yield:

$$jaroWinklerSim(s_A, s_B) = jaroSim(s_A, s_B) + l * p * (1 - jaroSim(s_A, s_B))$$

◀

Both the Levenshtein distance and Jaro-Winkler similarity focus on individual characters, making them well suited for identifying matching records in attributes with minimal differences. As shown in Figure 5.1, our experiments indicate that

12 https://anhaidgroup.github.io/py_stringmatching/v0.4.2/Levenshtein.html

Levenshtein similarity slightly outperforms Jaro-Winkler and additionally features an improved differentiation between ground truth labels (see Appendix A). Consequently, we take Levenshtein similarity forward.

### 5.1.2 Phonetic Measures

The **Soundex** algorithm [AA] evaluates string similarity from the phonetic perspective, matching words or strings on how they sound in the English language rather than comparing individual characters. Specifically, strings are mapped to an abstract representation in which homophones share the same code. Each Soundex code consists of the first letter of the string as well as a three-digit code representing the first few following consonants – vowels are entirely neglected.

In addition to Soundex, several other phonetic similarity measures have been proposed. Metaphone [Phi90], developed by Lawrence Philips, improves upon Soundex with a more sophisticated encoding algorithm. Philips' approach accounts for a wider range of pronunciation variations and was subsequently further refined. For the purposes of this thesis, we focus on the original Soundex algorithm and define the corresponding similarity measure as *soundexSim*.

▶ **Definition 5.2.** Given two strings $s_A$ and $s_B$ with their respective Soundex codes, we define:

$$soundexSim(s_A, s_B) = \begin{cases} 1 & \text{if } Soundex(s_A) = Soundex(s_B), \\ 0 & \text{otherwise.} \end{cases}$$

◀

The Soundex similarity measure primarily identifies homophones and requires matching strings to share the same Soundex code. Since its values are binary and not continuous, its effectiveness is limited and highly dependent on the specific string values (see Appendix A for details). In our matching scenarios, we found a significant variability in its differentiating ability, reflected by the large standard deviation in Figure 5.1. Hence, we exclude the Soundex measure going forward.

### 5.1.3 Token-based Measures

Contrary to character-based measures introduced above, the **Jaccard similarity** operates on a token level [Chr12]. Given two strings split into individual tokens (e.g., whitespace split), we compare common tokens against the entire set of tokens in both strings. Consequently, the measure puts no emphasis on the order in

which tokens appear but requires exact equivalence for common tokens (extensions exist that include an inner, character-based similarity measure between individual tokens).

In the context of this thesis, we compare two tokenization techniques:

- **Alphanumeric Tokenizer.** Given an input string, the alphanumeric tokenizer extracts the longest possible sequence of alphanumeric characters. In addition to whitespaces, it also splits at (and excludes) any non-alphanumeric character. We consider this tokenizer to be the more "traditional" option.
- **BERT Tokenizer.** With the recent advancements of language models, more sophisticated tokenizers were developed. The BERT tokenizer, built to cater the BERT family of language models [Dev+19], uses the WordPiece algorithm to transform a string into tokens. Specifically, each word, identified by white spaces and punctuation within a string, is split into as few tokens found in the WordPiece vocabulary as possible. As this tokenizer splits complex words into known subparts, we consider it to be the more "modern" option.

We tested both tokenizers and found that the BERT tokenizer usually matches or outperforms its strictly alphanumeric counterpart. Figure 5.1 demonstrates a slightly improved *F1 score*, although coupled with a slightly worse standard deviation. Due to the promising distinguishing ability of Jaccard similarity coupled with BERT tokenization, we select this technique to take forward.

### 5.1.4 Semantic Measures

All measures used so far focus on literal characters or tokens, entirely neglecting the semantic overlap between strings. Word2Vec [Mik+13], one of the earliest pre-trained embedding language model, addresses this limitation by capturing the meaning of individual words. Using whitespace tokenization, Word2Vec maps each token to a dense vector where semantically similar words reside close to one another. For this thesis, we utilize Word2Vec vectors pre-trained on the Google News dataset[13]. Although more recent embedding models, such as *st5-base*, may incorporate a more precise semantic understanding, we chose Word2Vec due to its distinct architecture from the embedding language models we subsequently intend to fine-tune.

To construct a similarity vector representation for a string from its individual word vectors, we first aggregate these vectors by calculating their mean, resulting

13 https://code.google.com/archive/p/word2vec/

in a single vector representation for the entire string (in alignment with the methodology of Zeakis et al. [Zea+23b]). Subsequently, the similarity value between both strings is then determined using cosine similarity. However, as the mean of the individual vectors is a lossy operation that reduces the semantic detail, this measure is most effective when applied to strings with a limited number of tokens (or words). Consequently, we exclusively apply this Word2Vec similarity measure to individual attributes rather than the aggregate value.

Our similarity measure is most effective where matching attributes contain similar (or identical) tokens that bear similar meanings. As shown in Figure 5.1, this measure performs slightly worse overall compared to token- or character-based measures. However, we choose to include it due to its strong performance in certain domain-specific scenarios, such as D10.

### 5.1.5 Textual Measures

To additionally address cases in which values are swapped between individual attributes, we include a similarity measure calculate on the **TF-IDF** vector of the aggregate value (see Section 4.2). The TF-IDF [SB88] measure was originally introduced in 1988 for information retrieval and weighs a word's importance for a document (or record in our case) both on its frequency within this document and how rare it is among all document. Hence, it is well suited as a textual measure for long strings (or more generic, for textual values).

To form a **TF-IDF similarity measure** for each pair of records, we calculate a TF-IDF vector over the 8192 most common trigram tokens found within the data sources and compare individual vectors using cosine similarity. As we focus on common tokens only, this measure benefits less from rare trigrams which, if included in both records, could be an indicator for a match. This effect is softened by utilizing trigram tokens, compared to language-centric tokenization options we used with token-based measures.

Our evaluation concludes a strong differentiating capability across all scenarios (refer to Appendix A for detailed results for each scenario). As demonstrated in Figure 5.1, it significantly outperforms all other measures in terms of mean *F1 score* and also exhibits the lowest standard deviation. Consequently, we select it as one of the most significant measures for creating the final weak label.

### 5.1.6 Numeric Similarity

Finally, we introduce a similarity measure for numeric attributes. Matching records upon numeric features is a dedicated research area, as explored in depth by Laskowski

in 2023 [Las23]. According to Laskowski, numeric data presents additional challenges in matching due to multiple valid representations of the same number, variations in measurement units, and various ways to assess numeric similarity (e.g., textually, by difference, or using numeric embeddings).

For the purposes of this thesis, we adopt a straightforward approach inspired by Primpeli & Bizer [PB20], calculating the absolute difference between two numeric values and normalizing their difference to yield a similarity value $numDiff \in [0, 1]$.

▶ **Definition 5.3.** Let $s_A, s_B$ be two numeric values from the same attribute. We calculate $numDiff(s_A, s_B)$ as:

$$numDiff(s_A, s_B) = \frac{|s_A - s_B|}{max_{(s'_A, s'_B)}\{numDiff(s'_A, s'_B)\}}$$

◀

Our evaluation across all numeric columns outlines very mixed results: While some scenarios, such as matching citation data on *year* in D4, achieve decent results, others do not exhibit any differentiating power on numeric columns (see Appendix A for further details). One potential cause for these results could be that the normalized difference measure is only applicable to some cases of (non-) matching attribute values. Hence, we exclude *numDiff* from further experiments and suggest to only include it in case the target scenario benefits significantly from the inclusion of numeric attributes.

### 5.1.7 Conclusion

Based on our experiments illustrated in Figure 5.1, we select Levenshtein, Jaccard/BERT, Word2Vec, and TF-IDF as our final set of similarity measures. As outlined earlier, these measures are collectively supposed to bear the potential to cover a wide variety of target domains and matching scenarios. Nevertheless, we acknowledge that this selection is inherently opinionated and can be further adapted to meet specific requirements, such as through the inclusion of numeric measures. Furthermore, these measures could be applied with greater customization, e.g., by differentiating between short and long strings.

It is crucial to emphasize that these evaluations rely on the existence of a ground truth annotation for each scenario, which is unavailable in truly unsupervised scenarios. Consequently, we prioritize a mixed ensemble of diverse measures rather than optimizing for specific scenarios.

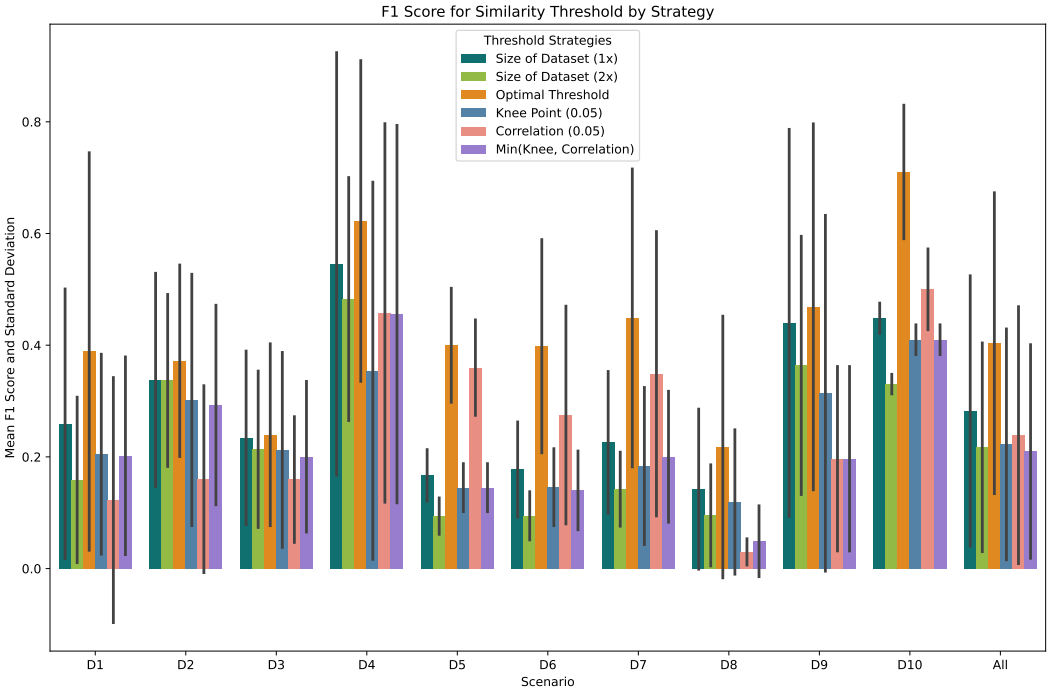


**Figure 5.2:** F1 score of similarity threshold according to strategy represented by mean (with standard deviation) for each scenario as well as combined for all scenarios. Additionally, the *F1 score* achieved with an optimal threshold is included. Higher is better. Elbow approach given as "knee".

## 5.2 Thresholds: From Continuous Values to Binary Labels

While the similarity measures we evaluated above enable us to extract similarity features for each pair in the candidate set, they do not provide a definitive statement: at which threshold should a similarity measure's continuous value be interpreted as a match or non-match?

Although determining such thresholds is not a prerequisite for the subsequent feature aggregation stage, it significantly influences its complexity. Aggregating binary labels is a well-understood task, extensively studied in the context of crowdsourcing and supported by decades of research (see Section 5.3). This enables our aggregation stage to build on existing methodologies. In contrast, the aggregation of continuous labels remains a relatively underexplored area.

Determining optimal thresholds for a given similarity measure is an inherent part

of entity resolution [KTR10]. For rule-based approaches, domain experts manually select the threshold [Wan+11], whereas machine learning-based methods estimate the optimal decision boundary during training. However, only a few publications exist that focus on estimating such thresholds independent of training data and solely upon the data itself. For dirty entity resolution on a single data source, dos Santos et al. propose a fully unsupervised approach based on profiling the resulting entity clusters [San+11]. Draisbach and Naumann analyzed the relationship between threshold selection and dataset size, concluding that the two are inherently dependent [DN13].

Nevertheless, we identified a lack of prior work addressing threshold estimation for clean-clean entity resolution. Consequently, we propose three approaches that build on existing concepts but incorporate novel ideas tailored to our specific application. The results of our evaluation are presented in two ways: First, we calculated each similarity features *F1 score* against the ground truth and compared the mean and standard deviation by matching scenario. The optimal threshold is included as a reference point. Second, we calculated the difference to the optimal threshold for each similarity feature.

### 5.2.1 Fixed Thresholds

The first technique is based on a straightforward observation: in clean-clean ER, the number of possible matches is inherently limited by the size of the smaller data source. In practice, we estimate that the true number of matches is often even smaller in most use-cases (e.g., see Table 4.1). Furthermore, we assume that there is no clean decision boundary between true matches and true non-matches anyway, and that the optimal threshold induces both false positives and false negatives.

As demonstrated in Figure 5.2, fixed thresholds with the size of the smaller data source (1x) and twice the size (2x) deliver decent *F1 scores* across most scenarios. More specifically, we observe once the size (1x) provides the best results on average, but performs poorly on more complex scenarios D5, D6, D7 from the movie domain. Utilizing twice the size (2x) never manages to outperform its sibling, due to additional false positives. Figure 5.3 confirms our observations and shows that the distance and standard variance to the optimal threshold vary significantly by scenario.

We acknowledge that this technique is heavily dependent on the size of the data sources, which might only coincidently correlate well with the actual count of matches. Furthermore, a poor similarity measure's optimal threshold requires additional flexibility around the placement of the threshold, which this technique

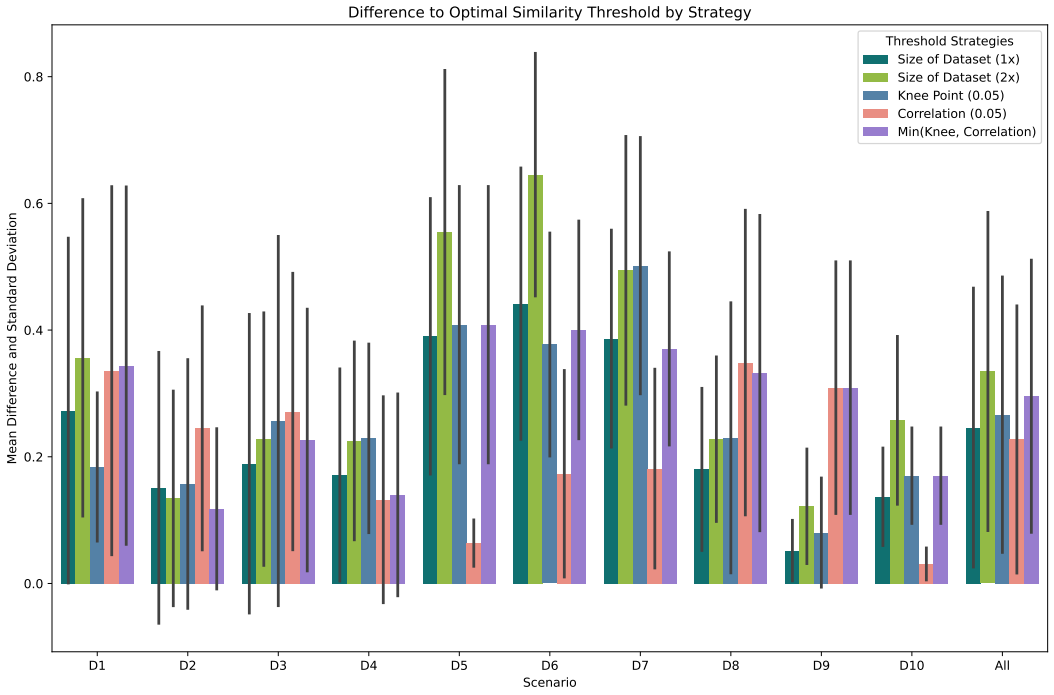


**Figure 5.3:** Difference of similarity threshold according to strategy compared to optimal threshold represented by mean (with standard deviation) for each scenario as well as combined for all scenarios. Lower is better. Elbow approach given as "knee".

cannot provide (i.e., selecting a higher threshold to limit false positives). To preserve generality, we therefore exclude this method from subsequent steps in our pipeline.

### 5.2.2 Elbow Approach

A well-known approach builds on a similar idea: as the set of true non-matches is magnitudes larger than that of true matches, lowering the threshold below the optimal value should cause a sudden increase in the count of pairs erroneously labeled as a match. This point of maximum curvature is commonly referred to as "take-off point", knee or elbow. For our use-case specifically, we search for elbow points (compared to knee points) as our graphs are constantly decreasing, concave functions. Figure 5.4 illustrates such a graph for scenario D5. To align our terminology with the Python libraries used, all evaluations and respective diagrams will refer to this approach as "knee".

In their paper *Kneedle*, Satopää et al. propose both a formal definition and a search algorithm for identifying elbow (or knee) points. The authors note that prior work lacks a clear definition, presumably due to the "system-specific" nature of the concept [Sat+11].

**The Kneedle Algorithm** identifies one or more possible elbow points on a given graph by locating points of maximum curvature. The algorithm is based on the idea that elbow points can be interpreted as local maxima if the graph is rotated such that the line connecting its start and end points becomes horizontal. Since a graph may contain multiple elbow points, the algorithm offers both offline and online modes: the offline mode performs a thorough search to identify the most significant elbow, while the online mode stops at the first elbow detected. As the algorithm may fail or return no elbow point, we address this case with a hard-coded threshold of 1.0.

For our use case, we utilize the offline mode and invert the graph so that the similarity threshold 1.0 marks the starting point of the algorithm's search. Additionally, the number of duplicate pairs cannot exceed the number of records in the smaller data source. Consequently, we can further restrict the search space to the portion of the graph where the count of pairs labeled as matches remains below the size of the smaller data source. The specific value for this restriction was determined experimentally and ensures proper termination if no meaningful threshold is found. Furthermore, it prevents the algorithm from being misled by curvature changes beyond the relevant part of the graph.

As shown in Figure 5.2, the elbow-based approach, despite our optimizations, consistently fails to outperform the other thresholding techniques. While it occasionally outperforms the correlation-based dynamic approach in certain scenarios,

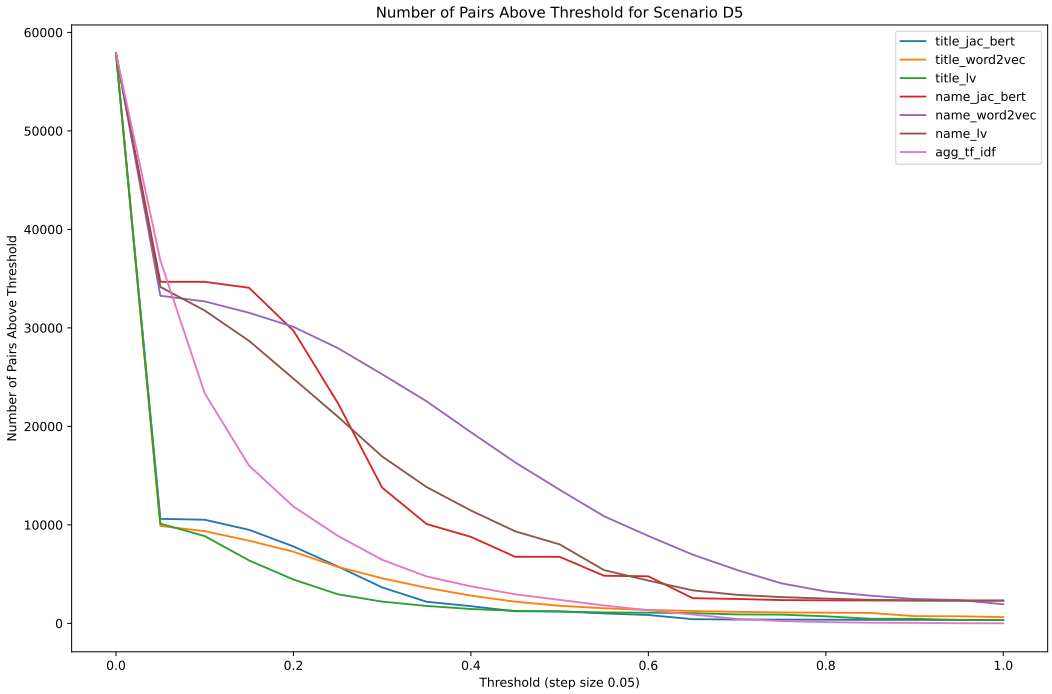


**Figure 5.4:** Number of pairs with a similarity value above a given threshold. The graph is cumulative if read from right to left. Each line represents one similarity feature.

the improvement is minimal. This limitation is likely due to its dependency on the cumulative graph, which is inherently linked to the nature of the underlying similarity measure.

Figure 5.3 underlines this observation, demonstrating that the approach struggles to define a threshold close to the optimal value in all but the simplest scenario, D1. One possible explanation for this behavior is that simpler matching scenarios produce clearer cumulative graphs, making the elbow point around the optimal threshold more apparent.

### 5.2.3 Correlation Approach

The correlation-based approach builds on the assumption that a good threshold produces binary labels that "look correct". Given a decent reference measure, such as a reliable similarity indicator or an incomplete ground truth annotation, an optimal threshold yields labels that are closely correlated to the reference labels.

Given the unsupervised nature of our pipeline, the options for such a reliable reference measure are fairly limited. To address this, we utilize the embedding similarities that are generated during the blocking stage. Under the assumption that the language model already performs quite well and exhibits deficiencies different from those of the similarity measures under evaluation, its values can serve as an effective reference measure.

Again, setting the threshold too low leads to a sudden increase in false positives, leading to a decrease in correlation. We expect this decrease to be more significant than the deficiencies of the embedding model itself. In contrast, false negatives caused by setting the threshold too high may be less apparent, as pairs around the decision boundary are inherently difficult to match. Consequently, the embedding model may provide inaccurate similarity values for these pairs, too.

On a technical level, we employ the Pearson correlation coefficient (PCC) [PG95] to test varying thresholds $t \in [0.05, 0.95]$ with a step size of 0.05. For each threshold, the binary labels are compared against the continuous embedding similarities, and the threshold with the highest correlation is selected.

During testing, we found that some similarity measures produced particularly low or negative correlations. To handle such cases, we define a minimum correlation threshold that a measure must meet; otherwise, it is assigned a value of 1.0.

With regard to the achieved *F1 score*, Figure 5.2 demonstrates the strengths of this approach: it outperforms the competition significantly on the movie scenarios D5, D6, D7 and particularly also on the challenging scenario D10, and generally provides decent results. Interestingly enough, it appears to struggle specifically with easier scenarios such as D1 and D2, where its predicted threshold is further away

from the optimal value compared to the elbow method (see Figure 5.3). Overall, we see a high variance in the difference to the optimal threshold. Despite this, the correlation approach manages to outperform its competition on average.

### 5.2.4 Combined Approach

During our testing, we observed that both the elbow and correlation approaches perform well in specific scenarios. To leverage their strengths, we additionally propose a combined approach that selects the smaller threshold returned by either method. This concept is based on the assumption that choosing the threshold too low is less likely, as it would result in a sudden increase in false positives. As a bonus, this combined approach can effectively handle cases where either approach returns a "muted" value of 1.0.

In terms of evaluation, the combined approach showed mixed performance (see Figure 5.2). For scenario D6, it even underperformed compared to its individual components. However, the combined method managed to demonstrate improvements over the worse of both in scenarios where either the elbow or correlation approach performed unexpectedly poorly, particularly for D1, D2 and D8. Overall, the combined approach demonstrates a decent level of generality and is consequently an effective thresholding strategy.

### 5.2.5 Conclusion

Based on our evaluation, identifying a single leading thresholding strategy remains challenging. A hard-coded threshold, while simple, lacks the flexibility to adapt to varying scenario properties, making it too dependent on simple data source characteristics. Among the dynamic methods, the correlation approach shows a slight advantage and particularly excels on more challenging scenarios.

While individual scenario analysis is helpful to understand each method's strengths and weaknesses, it conflicts with our goal of developing a generic and truly unsupervised pipeline. Conversly, only relying solely on the mean performance neglects critical edge cases, especially when an approach unexpectedly underperforms. As a result, we base our decision on the full picture and select the combined approach, as it effectively addresses underperformance and is expected to offer the best generality in practice.

## 5.3 Aggregation: Fusing Multiple Labels Into One

With thresholds selected and binary labels created, our next task is to aggregate these labels into a single decision for each pair: match or non-match. This aggregated label then serves as the target value during fine-tuning of the embedding model.

Luckily, the aggregation of multiple labels of varying accuracy (or reliability) has been extensively studied, particularly in the context of crowdsourcing. Platforms like Amazon Mechanical Turk[14] enable convenient access to a global workforce tasked with labeling training data. However, the accuracy of individual workers can vary significantly for various reasons, making the meaningful aggregation of multiple labels for each instance a critical step.

Similar challenges arise in other domains, where categorical data from sources with varying reliability must be combined. The approaches we evaluate are a mix of purpose-built algorithms and adapted approaches from related fields. For our use case, we configure all methods to not abstain from a final label and thus map such a label to 0 (non-match).

To evaluate our approach, we generated weak labels using the selected similarity measures from Section 5.1 and thresholds derived from the combined approach. Figure 5.5 already illustrates the performance of the aggregation methods, which is further detailed in the following section.

### 5.3.1 Flexible Majority Voting

The most straightforward approach to combining multiple binary labels is a majority vote. Here, we do not apply any weighting and consider each label with equal importance. Consequently, an individual similarity feature's accuracy is not considered.

▶ **Definition 5.4.** Let $F(p)$ be the binary feature vector for a pair $p \in P$. We define

$$majorityVote(p) = \begin{cases} 1 & \text{if } \sum F(p) \geq \lceil \frac{1}{2} \cdot len(F(p)) \rceil, \\ 0 & \text{otherwise.} \end{cases}$$

◀

To address situations where multiple similarity measures fail or label all samples as non-matches, we gradually lower the majority quorum until at least one pair

14 https://mturk.com

is labeled as a match. This added flexibility is necessary to account for situations in which the threshold selection fails with a value of 1.0, labelling all pairs as non-matches. Our mitigation strategy relies upon the assumption that at least one pair, after label aggregation with a subset of all similarity features, qualifies as a *match*.

Throughout our evaluation, as illustrated in Figure 5.5, majority voting consistently performs at or near the level of the best approach across almost all scenarios. The sole exception is scenario D7, where it delivers significantly worse results compared to Dawid-Skene (see Subsection 5.3.2). Despite this, its mean *F1 score* outperforms the competition, with no other "plain approach" (i.e., without added majority voting) achieving a mean *F1 score* within its standard deviation. This consistency, robustness, and generality make majority voting a particularly strong choice for our pipeline.

### 5.3.2 Dawid Skene Algorithm

Already in 1979, Dawid and Skene proposed an algorithm to aggregate labels found in patients' medical history files [DS79]. In their paper, they describe the use-case in which multiple clinicians may provide differing answers to the same question about a patient file and one has to aggregate those into a single statement. Similar to our use-case, the authors emphasize that their use-case has no true response available to compare against. On a technical level, their approach is based on approximating the maximum likelihood estimates of the parameters with the reliability of each source. To calculate the optimal parameters, an iterative optimization algorithm is applied that executes until it converges. Dawid and Skene state that their approach is slow, but in exchange provides accurate results.

Since the initial publication, the algorithm has been widely used to aggregate crowdsourced labels. Various publications exist that either apply or optimize the original algorithm, such as the *Fast Dawid-Skene* [SRB18] project. This novel approach preserves the idea of expectation maximization but achieves a speed-up of up to 8x.

Our implementation is based on the original version by Dawid and Skene and utilizes the *crowdnalysis*[15] library. To achieve compatibility, we first map our feature set to a crowdsourcing challenge and similarly revert this mapping afterward.

The Dawid-Skene aggregation approach doesn't manage to show consistent performance. Much rather, its only strong scenarios are D2, D4 and D7 where it even outperforms the competition (see Figure 5.5).

**15** https://github.com/Crowd4SDG/crowdnalysis

Besides the original algorithm, we also tested a second approach, where we apply Dawid-Skene on all pairs that were labeled as "match" by majority voting. The core idea here is to further refine the decision boundary and improve upon precision by filtering false positives. Our testing shows stronger performance than both methods individually on scenarios D4 and D8, but generally a similar picture as Dawid-Skene by itself. Interestingly, the performance on D1 is 0 which comes as a surprise since majority voting (and a similar combination for Snorkel) provides the best results. We conclude that the Dawid-Skene algorithm cannot provide strong enough results for our use-case, independent of whether it is applied by itself or combined with majority voting.

### 5.3.3 Data Fusion Technique

In the field of data fusion, researchers face a similar challenge: combining data from multiple sources with varying accuracy into a single value. Dong et al. describe an algorithm "that finds true values from conflicting information when there are a large number of sources [...]" [DBS13]. Their approach estimates both the source and individual value accuracies with a basic Bayes model, which assumes values to be categorical and that all false values have an equal probability of being presented. Conceptually, they address a challenge similar to that of Dawid-Skene.

For our aggregation purposes, we exclusively deal with binary values which meet the categorical requirement and naturally provide equal probability, as only a single false value is possible. In the following, we refer to a source as a feature (i.e., a similarity measure applied to a column) and simplify the formulas for binary labels. To begin, we model the accuracy of an entire feature in providing correct values.

▶ **Definition 5.5.** Let $F$ be a feature, $P$ the list of candidate pairs to assess, $P(F)$ the binary values of $F$ on $P$ (i.e., whether a $p \in P$ is a match according to $F$) and $Prob(p)$ the probability for one particular decision to be true. We define the overall feature accuracy $Acc(F)$ as

$$Acc(F) = \frac{\sum_{p \in P(F)} Prob(p)}{|P(F)|}$$

◀

Using this definition, we can now express the probability of a single value belonging to a specific label class. Since our use-case focuses exclusively on binary labels with two classes, pos (match) and neg (non-match), the probability of the

positive class being correct, $Prob(p = \text{pos})$, equals the probability of a pair being an actual match.

▶ **Definition 5.6.** With $\mathcal{F}(p)$ as the set of features with the same label assigned to $p$ (i.e., either match *pos* or non-match *neg*), we define the confidence $Conf(p)$ of assigning this particular label to a pair $p$

$$Conf(p = \text{pos or } p = \text{neg}) = \sum_{F \in \mathcal{F}(p)} log\left(\frac{Acc(F)}{1 - Acc(F)}\right)$$

Consequently, the probability of a pair's label to be true and hence for the pair to be a match is

$$Prob(p = \text{pos}) = \frac{2^{Conf(p=\text{pos})}}{2^{Conf(p=\text{pos})} + 2^{Conf(p=\text{neg})}}$$

◀

The definitions of feature accuracy and label accuracy form a cyclic dependency, which converges through iterative computation. We initialize the feature accuracies by uniformly sampling from $[0.6, 0.9]$. In our testing, the algorithm usually converges in fewer than 100 iterations, with computations deemed as "failed" after more than 1000 iterations.

The data fusion method delivers overall consistent results and occasionally even outperforms the competition (see Figure 5.5). Although its average *F1 score* makes it the runner-up among plain approaches, its performance in certain edge cases, such as scenario D8, raises concerns when developing a generic approach. As a result, we exclude this method from further analysis.

### 5.3.4 Snorkel's Label Aggregation

Finally, we introduce the Snorkel framework by Ratner et al. [Rat+17] which is purpose-built for programmatic weak supervision (PWS, refer to Section 3.2 for further details). At the core of the framework is the idea to allow for training a machine learning model without a single reliable label source, but rather on the output of a set of unreliable labeling functions. In practice, Snorkel expects its users to come up with labelling functions by themselves, impacting accuracy and independence of the final label set, but allowing for cheap and quick creation of a potentially large label set. Given an input sample, such a labelling function either provides a label as output or returns $-1$ to abstain. The Snorkel framework can deal with a diverse set of labelling functions, e.g., one function with high accuracy that applies to a smaller subset and a less reliable one which outputs labels for all

samples. Besides requiring more than 50% accuracy for each labelling function, there are no further restrictions that apply.

From a technical standpoint, Snorkel is a combination of a generative and a discriminative model. While the generative model learns feature accuracies and their correlations, the discriminative model aims to generalize to all samples for increased coverage and robustness. To outperform a basic majority model, Snorkel includes a fallback mechanism that is supposed to activate when the majority model is expected to yield higher success rates.

A common challenge for machine learning models in entity resolution is the inherent label imbalance between matches and non-matches. Since the vast majority of instances to be aggregated are labeled as *non-matches*, models often overfit by labeling all instances as *non-matches*. To mitigate this issue, many machine learning libraries include parameters that allow to specify the expected class imbalance. These parameters influence the loss function by amplifying the loss of the underrepresented class. Our implementation estimates the expected label imbalance using the results of majority voting and provides this information to Snorkel.

Despite this additional parameter, Snorkel did not deliver convincing results out of the box (see Figure 5.5). This outcome was unexpected, given that the framework was provided with a class imbalance estimation. Nevertheless, the underlying models were unable to reliably learn to differentiate between matches and non-matches. We hypothesize that this limitation is related to the extreme class imbalance, which may surpass the corrective capacity of class balancing techniques. Further research is required to determine how Snorkel can be effectively adapted for the aggregation stage of our pipeline.

Additionally, we tested a combined approach similar to the setup with Dawid-Skene. The results were mixed, with the combination performing close to majority voting in most scenarios. Interestingly, certain scenarios such as D3 and D7 demonstrate that applying Snorkel to the results of majority voting can lead to an uplift in performance. However, other scenarios, such as D6 and D9, highlight the scenario-dependent nature of this effect.

Overall, the mean performance of the combined approach remains below that of majority voting, and its performance on majority voting's weakest scenario, D8, is worse. Consequently, we exclude both Snorkel-based approaches from the next stages of the pipeline.

### 5.3.5 Conclusion

In this subsection, we introduced various aggregation methods. Surprisingly enough, more complex approaches such as Dawid-Skene or Snorkel demonstrated

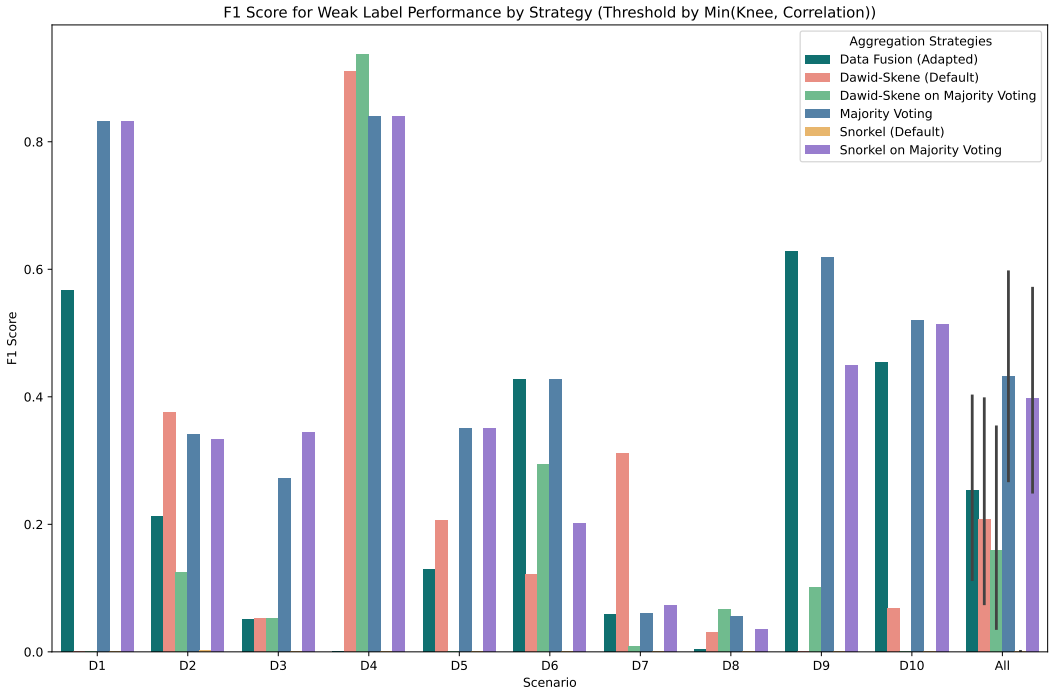


**Figure 5.5:** Evaluation of aggregation strategies by *F1 score* on all scenarios. Weak labels generated as described in Section 5.1 and thresholds calculated with combined approach.

an overall poor performance, with default Snorkel often labelling all pairs as non-matches. Using both methods as post-processing operations upon the results from majority voting significantly improved their own performance, at least for some scenarios, and brought Snorkel to the second rank in mean *F1* performance (among all methods). Nevertheless, the consistent performance of majority voting on all but one scenario, D7, makes it a prime candidate for the aggregation stage of the pipeline. This choice is backed by the fact that no other plain approach achieves a mean *F1 score* which is within the majority voting's standard deviation. Hence, whether we evaluate individual scenarios' *F1 scores* or not, majority voting provides the most balanced and overall best aggregation performance among all methods.

**A Word on Downstream Influence.** As highlighted in the conclusion of the thresholding section (see Section 5.2), generality is a critical aspect of our pipeline. While the evaluation of the combined thresholding approach (see Figure 5.5) allows for aggregation methods to demonstrate consistent performance among all scenarios, running the same experiments with only the correlation approach reveals a starkly different picture. Instead of consistent results, Figure 5.6 illustrates significant variance among the individual scenarios, particularly in the "easy" cases such as D1 and D2. This discrepancy underscores how a lack of generality in one stage can substantially impact the downstream performance, especially given the unsupervised nature of our pipeline.

## 5.4 Sampling: Quality over Quantity

During early testing, we observed that "the more, the better" does not apply to training sample selection. Instead, carefully selecting a meaningful subset of samples yielded the best performance. The reasoning behind this is twofold.

First, research around fine-tuning large language models, such as by Chen et al. [Che+23], demonstrates that a small, high-quality, and diverse subset of all samples, can outperform training on the entire dataset. While their motivation primarily focuses on cost-reduction and training efficiency, recent publications extend these concepts further. Shen hypothesizes that "data selection [...] should focus on picking demonstrations that reflect human style the most" [She24]. Although the embedding model we focus on in this thesis is less complex than modern LLMs, focusing on fewer high-quality samples is preferable to a potentially larger, less accurate set that includes lower-quality instances. For our training setting, quality refers to both label reliability and meaningfulness of the textual content.

Second, the quality of the entire training set poses a critical challenge in our use case. On the one hand, entirely unsupervised weak label generation is inherently

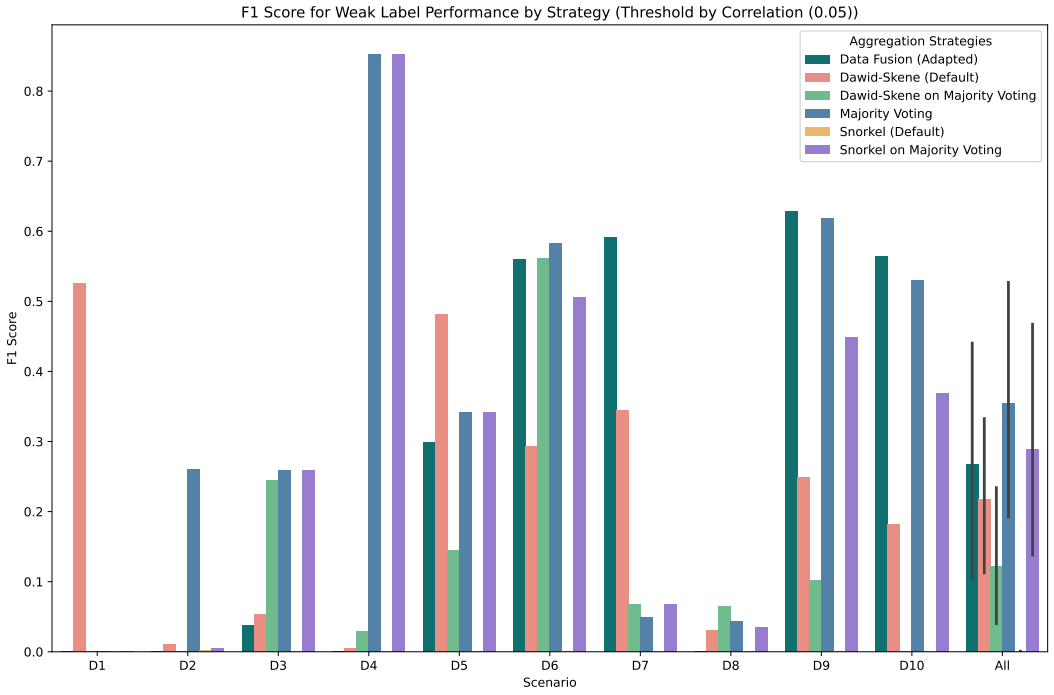


**Figure 5.6:** Illustration of the downstream impact of threshold selection on aggregation strategies, using thresholds determined by the correlation approach. Weak labels are generated following the methodology outlined in Section 5.1.

difficult and, as illustrated in Figure 5.5, far from producing perfect labels. On the other hand, the candidate set for weak label generation remains large, where easy negatives far outweigh meaningful instances. Quite intuitively, showing easy non-matches to a model has limited impact on its ability to handle hard cases effectively.

Consequently, we tested multiple approaches to sample a smaller training set from the full weak-labeled training set. The evaluation of these sampling approaches is closely tied to the training process itself, including the choice of objective function during fine-tuning. Thus, we provide an overview of each approach in this section and defer their effectiveness evaluation to the following section, which focuses on the fine-tuning aspects of our pipeline.

### 5.4.1 Individual Pairs

The first group of sampling techniques focuses on choosing individual instances independently of one another.

- **Truly Random.** The most straightforward sampling approach would be to sample uniformly across all weak-labeled instances. As stated above, this technique causes a significant underrepresentation of samples labeled as match within the subset that is usually less than 1%, potentially, with many easy non-matches. Consequently, we exclude this technique from further experiments.

- **Equally Random.** As a baseline, we sample uniformly random from both label classes such that 50% of our subset is of label *match* (and *non-match*, respectively). Thereby, we achieve a higher likelihood to choose meaningful instances in a diverse set that improve the model's performance on hard cases. Nevertheless, we risk also choosing false labels (i.e., hard cases with an incorrect weak label) that could have a negative impact on its matching competency.

- **Most Certain.** The other extreme is to choose weak labels that are most certainly correct. Intuitively, these reside on the extremes of the spectrum, either with the highest or lowest similarity. Sampling equally from both ends yields certain, but likely very easy, non-matches as well as particularly confident matches. Nevertheless, we effectively reduce the risk of including false labels and provide the model with a high-quality, yet "easy", training set.

Beyond the sampling techniques proposed here, alternative measures can be employed to rank and select pairs. In their work, Laskowski and Sold [LS23] present a method for carefully selecting pairs from a potentially large entity resolution result set for manual inspection. Their approach builds on the informativeness measure introduced by Christen et al. [CCR20], designed to select pairs for manual labeling in active learning scenarios. The objective of identifying a representative and informative subset closely aligns with our requirements for the training set.

### 5.4.2 Groups of Records

Instead of sampling individual pairs, groups of connected instances can be sampled, too. The core idea is to select an anchor record (not a pair) along with multiple matching and non-matching pairs involving this record. In the following Chapter 6, we introduce the *Triplet Loss* and *Multiple Negatives Ranking Loss (MNR Loss)* objective functions for fine-tuning. The *Triplet Loss* requires one anchor record, one matching record and one non-matching record, while the MNR Loss utilizes multiple non-matching records. Accordingly, we now outline the process for sampling instances to meet these requirements.

As a first step, we sample $n$ meaningful instances among the set of pairs weak-labeled as matches. The selection thereby occurs "farthest-first", initially compared to a randomly chosen instance, and afterward iteratively compared to the sum of all similarities to all selected pairs. As a distance measure, we use the cosine similarity between the similarity feature vectors (with continuous values as generated in Section 5.1) of the pairs. Thereby, our distance measure enables us to select a diverse set of pairs, i.e., pairs who demonstrate varying similarity values and thus are likely to form a diverse set. For each pair in our subset, we assume the left record (from data source A) to be the anchor, without loss of generality.

Next, we utilize the semantic similarity values we computed during the blocking phase to find semantically similar records with whom the anchor forms non-matching pairs according to our weak labels. Here, we choose negatives with descending similarity until we collected enough negative instances for our anchor record. Depending on the objective function, the instances we selected then serve as a triplet (anchor, positive, negative) or a group with multiple negative pairs.

Regardless of the selection approach and types of instances sampled, the output of this stage serves as the training data for fine-tuning the embedding model. As discussed, the strengths and limitations of each sampling technique are closely tied to the training setup and process.

## 5.5 Retrospective & Summary

Within this section, we outlined the process by which our pipeline creates weak labels. Starting with a candidate set, we first generated similarity features using various similarity measures. This resulted in a similarity vector for each pair, where each attribute is represented by one or multiple similarity feature values, alongside additional pair-wide features. Through a detailed evaluation, we selected a final, albeit opinionated, set of features to represent the candidate pairs.

As these features are continuous values, we evaluated multiple approaches to define a similarity threshold automatically and entirely unsupervised. Surprisingly, static techniques relying on the scenario-specific size of the smaller data source yielded strong results. Nevertheless, we selected a dynamic, combined approach built on both elbow point detection and correlation maximization, which demonstrated competitive performance across various scenarios.

Next, we addressed the aggregation of multiple labels per pair into a single binary label, *match* or *non-match*, that can be used in fine-tuning. Complex techniques, while theoretically promising, exhibited weak average performance and inconsistent results depending on the scenario. In contrast, a simple majority voting outperformed other methods across most scenarios, delivering reliable results.

Finally, we introduced various sampling techniques that select either individual pairs or groups of records. In the context of this thesis, each category is represented by one or multiple sampling approaches, with the flexibility to implement additional ideas and concepts. Similarly, the other sections present an initial set of approaches that can be extended with further research. While the individual stages of weak label creation are interconnected and influence one another (e.g., as discussed in Subsection 5.3.5), they function as modular components that can be easily replaced or adapted to new methods and techniques.

**Matching Solely with Weak Labels.** In Section 5.3, we evaluate the entire weak-labeled candidate set against the ground truth, mirroring the assessment of an entity resolution framework's output. This demonstrates that our weak label creation can function as a standalone matching system, particularly since it operates in an entirely unsupervised manner. While its performance is not yet competitive with ZeroER by Wu et al. or the pre-trained embedding model (see Section 4.1, Figure 5.5), our approach shows robustness across various scenarios. Moreover, its fully modularized design allows individual components to be easily interchanged, positioning it as a strong candidate for further research around unsupervised entity resolution.

# 6 From General to Specific: Fine-Tuning Embedding Models

In Chapter 5, we discussed the process of creating a weak label training set, which resulted in either individual pair instances or groups of records. Using these samples with their respective weak labels, we can now proceed to fine-tuning embedding models.

As a first step, we introduce objective functions and explain their characteristics. Next, we summarize the overall fine-tuning process and outline the hyperparameters we selected. Finally, we conduct a comprehensive end-to-end evaluation of our pipeline on all scenarios described in Section 4.2.

**Fine-tuning Language (Embedding) Models.** Pre-trained language models are typically trained on a broad and diverse corpus of data. With their pre-training focused on a generic understanding of various domains, these models often demonstrate notable performance on a wide range of tasks – particularly those included in their pre-training objectives. However, task-specific fine-tuning can further enhance their capabilities for a specific use case. In our context, the model may already incorporate a reasonable understanding of which features determine whether two film entries form a match or non-match, depending on the dominance of relevant pre-training learnings in its output. By introducing the model to a small set of use-case-specific examples during fine-tuning, we can increase the likelihood that relevant learnings influence the output effectively.

## 6.1 Fine-tuning: Teaching Effectively

As outlined in Section 3.1, the *st5* embedding model family is derived from a generic language model specialized for the semantic textual similarity (STS) task. Given an input sequence, the model outputs an embedding vector, where vectors of semantically similar sequences exhibit a high similarity to one another.

Fine-tuning, like other forms of model training, requires an objective function. This function computes a loss based on the expected and actual model output, which is then used to optimize the model's weights toward the target behavior. We distinguish between two approaches for defining this target behavior: one based on individual pairs and another utilizing groups of records. The following subsection introduces each approach and highlights their key characteristics.

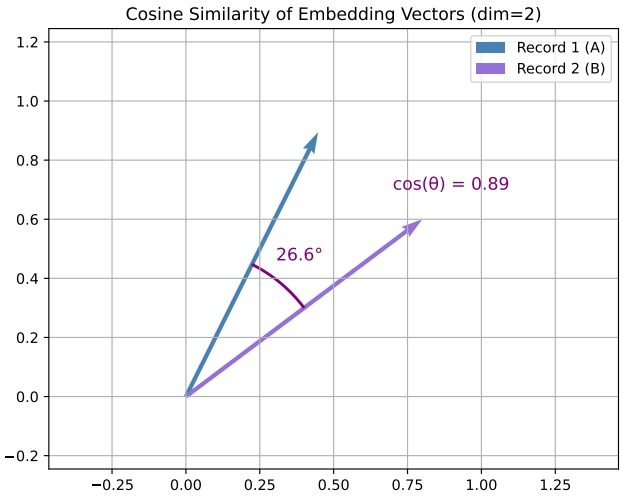


**Figure 6.1:** Visualization of a pair of records from data sources A and B. Both vectors are normalized, with the angle between them provided in degrees and as the cosine similarity value.

### 6.1.1 Objective Function for Individual Pairs

For pairs of records, the expected behavior corresponds to the ideal class label for the pair, match or non-match, represented as 1 or 0. The model generates an embedding vector for each record, and their similarity value $\in [0, 1]$ serves as the pair's predicted label. Figure 6.1 illustrates two record vectors along with their cosine similarity value. For the purposes of this thesis, orthogonal vectors with a similarity value $< 0$ are mapped to 0.

As discussed in Section 3.1, the model's initial training was designed to optimize its vector output such that semantically similar text fragments are mapped to vectors with high similarity values. Similarly, our fine-tuning objective aims to maximize the similarity for likely matches and minimize it for non-matches.

The Sentence Transformers library already provides several objective (or loss) functions[16] to optimize the similarity value for individual pairs against an expected similarity score. The most basic loss function, *CosineSimilarityLoss*, minimizes the mean squared error (MSE) of the difference between the predicted similarity and the target label. Subsequent research by Li and Li proposes the *AnglELoss* [LL24], which aims to mitigate some of the deficiencies of *CosineSimilarityLoss* and other work in this space.

The core idea of *AnglELoss* is to "optimize the angle difference in complex space"

16 https://sbert.net/docs/package_reference/sentence_transformer/losses.html

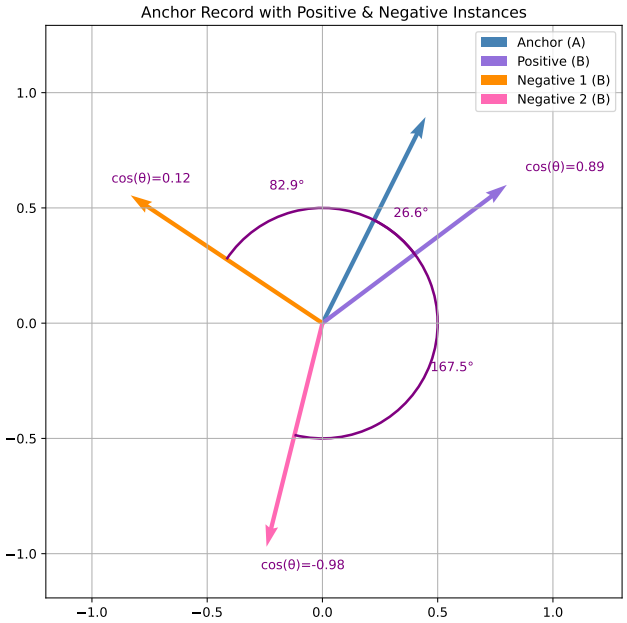


**Figure 6.2:** Visualization of a group of records from data sources A and B. The anchor record is the reference for all angle calculations.

to circumvent the low gradients of the cosine function around $n * \pi$ with $n \in \mathbb{N}_0$, thereby providing a stronger training signal. Their results demonstrate an increase in STS performance across various combinations of language models and objective functions, particularly also in comparison to the *CosineSimilarityLoss* (represented by *CoSENTLoss*, one of its successors). Based on these findings, we select *AnglELoss* as the objective function for pair-based fine-tuning.

### 6.1.2 Objective Function for Groups of Records

In contrast, defining the expected behavior for groups of records is less intuitive. Given an anchor record, the positive example is expected to have a particularly high similarity to the anchor, while all negative examples should exhibit a very low similarity. Figure 6.2 illustrates this setup for vectors with two dimensions. For this thesis, we sample the anchor record from data source A, while the positive or negative records are sampled from data source B.

The most straightforward training setup, using *TripletLoss*, involves triplets, each consisting of an anchor, a positive and a negative example. The objective of this

setup is to maximize the similarity between the anchor and the positive example while simultaneously minimizing the similarity to the negative.

By incorporating multiple negative examples, the fine-tuning objective shifts from an individual pair level to a more complex multi-class classification problem. In other words, the goal is to map the anchor record to one of the records from the subset of the other data source. This setup includes one correct class (i.e., the positive example forms a matching pair with the anchor) and multiple incorrect classes. A similarity measure, such as cosine similarity, acts as the comparison metric by which the most likely class is identified. As a result, the model learns to maximize the similarity between the anchor and the positive example while minimizing the similarity with all negative examples.

In 2017, Henderson et al. [Hen+17] introduced a concept to train with multiple negatives for tasks like STS, which forms the foundation of the *MultipleNegativesRankingLoss* in the Sentence Transformers library. Their initial use-case was a smart reply recommendation for Google's Gmail service, where this approach demonstrated a significant improvement over models trained with a binary objective function (i.e., Sigmoid loss). Given a set of positive pairs $(a_i, p_i)$, the objective function automatically constructs negative pairs $a_i, p_j$ for $i \neq j$. In other words, all other right-hand records provided are automatically used as negative examples.

▶ **Definition 6.1.** For a single training batch, let $\mathbf{x}$ be a set of anchor records and $\mathbf{y}$ the set of their positive partner records. We define the objective function $\mathcal{J}$ to minimize as:

$$\mathcal{J}(\mathbf{x}, \mathbf{y}, \theta) = -\frac{1}{K}\sum_{i=1}^{K} \log P_{\text{approx}}(y_i \mid x_i) = -\frac{1}{K}\sum_{i=1}^{K}\left[S(x_i, y_i) - \log \sum_{j=1}^{K} e^{S(x_i, y_j)}\right]$$

◀

In 2024, students from Stanford University [CAG24] conducted a thorough analysis of how this objective function can be used for fine-tuning models from the SentenceBERT family. Their findings indicate a significant increase in performance across various STS tasks and suggest the viability of this objective function for fine-tuning embedding models.

While the original approach by Henderson et al. is applicable to clean-clean entity resolution, where each record from data source A is matched with at most one record from source B, it does not fully leverage the available data. Positive examples of other anchor records usually represent straightforward negatives to the anchor at hand, limiting the training benefit. Fortunately, the Sentence Transformers library allows for the manual inclusion of hard negatives alongside each training pair.

By incorporating both hard, manually sampled negatives and the easy negatives automatically constructed from other pairs in the training set, the model is fine-tuned to minimize similarity for a diverse set of negative examples. This approach provides the model with a comprehensive view of the task, enabling it to extract as much knowledge as possible from each batch of samples.

### 6.1.3 Fine-tuning Essentials: Hyper-Parameters & Training Workflow

Based on the two approaches outlined earlier, we now proceed with the fine-tuning process using the Sentence Transformers library's built-in trainer. During this process, various parameters – commonly referred to as hyperparameters – are optimized. While identifying optimal values for these parameters could yield the best possible results, we limit our efforts to understanding whether fine-tuning has a positive impact, rather than achieving the highest performance gains.

This subsection introduces the most important hyperparameters and provides guidance on how they could be further optimized. Here, the focus is to establish a foundation for evaluating our pipeline's effectiveness, leaving room for future work to explore more advanced optimization strategies for the fine-tuning process.

**Total Number of Samples.** As discussed in Section 5.4, the number of instances in the training set is essential, as choosing either too few or too many training samples can impact performance and training effectiveness. In addition, when training on groups of records, fewer total samples are needed, as the MNR objective function implicitly reuses examples from other anchors as negatives for the anchor at hand. Plain pair instances, on the other hand, require further consideration into how many samples should be shown during training.

**Samples per Batch.** As the number of samples increases, the memory consumption may exceed the available capacity. We opt to retain the default batch size of 8 recommended by the authors of the Sentence Transformers library. For the MNR objective function in particular, the batch size also determines the number of negative examples available for each anchor.

**Learning Rate.** During the back-propagation stage for each batch, the loss is computed with the objective function, and the respective gradients are used to update the network weights. The learning rate defines how much each weight should be altered. In our implementation, we use the linear learning rate scheduler provided by the Transformers library. Starting from on a predefined value, the learning rate gradually decreases throughout the training process. This approach ensures that the model can converge effectively, making "smaller" weight adjust-

ments as it approaches the ideal values. Without such a scheduler, the weights may oscillate around the optimal value, preventing proper convergence. The Sentence Transformers library suggests a default initial learning rate of $2^{-5}$.

**Total Training Epochs.** The total number of epochs is closely linked to the choice of learning rate. An epoch represents one complete pass through the training dataset (in our case, processed in batches of 8). For our use-case, it is crucial to select a sufficient number of epochs to ensure that training continues until fine-tuning converges. The loss produced in each epoch serves as a good indicator of the model's progress: if the loss continues to decrease, additional training epochs are likely beneficial. However, training for too many epochs can harm the model's performance by causing it to overfit the training data, particularly to incorrect weak labels. To maintain generalizability across all experiments, we fix the number of training epochs at 20.

**Training Process.** With regard to the training process, we do not split the available samples into separate validation or test sets. This decision is based on the fact that the weak labels we created do not serve as a reliable evaluation baseline for assessing the model's matching performance. Counterintuitively, we even want the model to not learn too excessively from a particular poor weak label set or to overfit to bad samples. The primary indicator of training performance is the decrease in loss computed by the objective function, and how this value evolves from epoch to epoch.

## 6.2 End-to-End Evaluation: Our Methodology

The primary objective of this thesis is to demonstrate that language models fine-tuned with weak labels, generated by our pipeline, can outperform their solely pre-trained counterparts. Accordingly, our evaluation follows the same evaluation workflow for unsupervised entity resolution as the baseline results reported by Zeakis et al.[Zea+23b] (see Section 4.1 for details). Similar to their workflow, we do not employ any blocking and directly apply the UMC clustering algorithm to all possible record pairs.

The key adaptations include replacing the pre-trained base model with our fine-tuned version of the *st5-base* model, using cosine similarity as the distance measure between embedding vectors. The latter choice aligns with the objective functions used in our fine-tuning process, which specifically optimize for cosine similarity. For the training setup, we evaluate four training strategies.

- **Truly Random Pairs (TRP).** Using truly random sampling, we select 128 pairs out of the candidate set and optimize the model using the *AnglELoss*

objective function (see Subsection 6.1.1). The learning rate is initialized to $2 * 10^{-6}$, and the model fine-tuned for 20 epochs.

- **Equally Random Pairs (ECP).** To improve the diversity of matches and non-matches, we sample equally from either weak label class. All hyperparameters, including the learning rate and number of epochs, remain identical to those used for the *Truly Random Pairs* strategy.
- **Most Certain Pairs (MCP).** This strategy selects pairs from either extreme of the aggregated similarity values, representing the most certain pairs for the match and non-match classes. Again, these pairs are fine-tuned using the same parameters as the *Truly Random Pairs* strategy.
- **Anchored Record Groups (ARG).** As detailed in Subsection 6.1.2, the MNR loss allows each training sample to consist of an anchor record, a positive example, and a set of hard negatives. We first sample 32 positive pairs and select an additional 15 hard negatives per record group. The model is then fine-tuned with an initial learning rate of $2 * 10^{-6}$ for 20 epochs.

**Hardware Capabilities.** The experiments part of this evaluation were conducted on the *Digital Health Compute Lab* which is part of *Scientific Compute Infrastructure*[17] at Hasso Plattner Institute, Potsdam, Germany. The host systems are equipped with two AMD Epyc 7543 processors, 1024GB of system memory and provide multiple Nvidia A40 GPU accelerators with 48GB of graphics memory each. An individual experiment receives dedicated, constricted access to 16 cores, 350GB of main memory and a single Nvidia A40 GPU. While the data sources initially reside on a network file system, this potential bottleneck is negligible as the first pipeline step loads all data into system memory.

**The Framework.** All concepts discussed in this thesis are also implemented in Python[18]. Our framework provides an abstraction above the underlying operations and can be controlled through a set of high-level functions. The code in Listing 6.1 outlines the core stages of the fine-tuning process and highlights the core parameters for each stage of our pipeline. Whenever possible, our implementation executes computations on the GPU accelerator to benefit from parallelized batch execution. The source code is publicly available for download[19] and serves as a decent foundation for further research and development around our pipeline. We acknowledge that the current implementation is primarily focused towards research purposes and requires additional optimizations to be applicable in practice.

17 https://hpi.de/forschung/hpi-datacenter
18 https://python.org
19 https://dl.fsold.de/master-thesis-fs2025/unsupervised-er-fine-tuning-sourcecode.zip

```python
from framework.pipeline import Pipeline
from sentence_transformers import losses

pipeline = Pipeline(
    scenario_key='D1',
    experiment_name='exemplary_run_001',
    model_name='sentence-transformers/gtr-t5-base',
)

pipeline.prepare_scenario(
    blocking_method='index_FlatIP',
    index_dimensions=768,
    nearest_neighbours=5
)

pipeline.stage_1_create_weak_labels(
    measures=['lv', 'jac_bert', 'word2vec', 'agg_tf_idf']
)

pipeline.stage_2_apply_thresholding(
    threshold_strategy='pearson_knee_min',
)

pipeline.stage_3_aggregate_labels(
    aggregation_strategy='majority'
)

pipeline.stage_4_sample_training_set(
    num_samples=128,
    strategy='farthest-first-positive',
    n_let_negatives=12
)

pipeline.setup_finetuning(
    loss_fn=losses.MultipleNegativesRankingLoss,
    num_train_epochs=20,
    learning_rate=2e-5
)

pipeline.run_finetuning()

pipeline.do_evaluation(
    use_cosine=True
)
```

**Listing 6.1:** Exemplary execution of the entire pipeline for scenario D5 with ARG and recommend defaults for all parameters. Source code implemented in Python.

**Evaluation Metrics.** The final matching result is, as implemented by Zeakis et al. [Zea+23b], computed at various thresholds $\delta$ and evaluated using *precision*, *recall*, and *F1 score* (see Section 2.1). The overall matching performance of an experiment is given by the metrics at the threshold $\delta$ with the highest *F1 score*. As a reference point for *F1* performance, we compare *st5-base* fine-tuned with our pipeline against the pre-trained base models *st5-base* and *st5-large* (see Section 4.1). Throughout this evaluation, our pipeline always fine-tunes the *st5-base* model independent of the reference model it is compared to.

## 6.3 Hard Numbers: How did we do?

As outlined, our experiments aim to compare various training strategies and their respective hyper-parameters, focusing on combinations that are particularly promising or impactful. To ensure generic applicability of our pipeline, we focus on evaluating only a few hyper-parameter settings for each strategy, leaving most values at their defaults. The primary goal of this section is to assess the sampling strategies *Truly Random Pairs (TRP)*, *Equally Random Pairs (ERP)*, *Most Certain Pairs (MCP)*, and *Anchored Record Groups (ARG)* (as introduced in Section 6.2). Each experiment consists of a specific strategy and a set of hyper-parameters, namely *Samples (S)*, *Epochs (Ep)* and *Learning Rate (Lr)*. Table 6.1, located at the end of this chapter, provides a detailed overview, contextualizing our experiments alongside the pre-trained models *st5-base* (*BASE*) and *st5-large* (*LARGE*). Through these experiments, we aim to investigate (I) which strategy yields the best performance, (II) whether optimizing hyper-parameters significantly influences results, and (III) how fine-tuning compares to using pre-trained models alone.

**(I) Training Strategies.** For *Anchored Record Groups (ARG)* sampling, we tested two configurations: 100 anchors with 15 negatives each, and 128 anchors with 12 negatives each. During testing, the configuration with 15 negatives failed for scenario D6 due to exceeding the memory capacity of the Nvidia A40 GPU used (see Table 6.1). These failed runs are recorded with an *F1 score* of 0.0, which impacts the mean performance for both affected experiments (see Figure 6.3).

The *Multiple Negatives Ranking* loss amplifies memory usage, as it utilizes negatives of other anchors as additional negatives to the anchor at hand, resulting in a sharp increase in memory consumption. A practical solution is to decrease the number of negatives per anchor, an option also applicable in practice which simultaneously delivers strong results in our evaluation.

In contrast, the weakest performance is observed with *Truly Random Pairs (TRP)* sampling. Due to the label imbalance in the training set, with potential matches

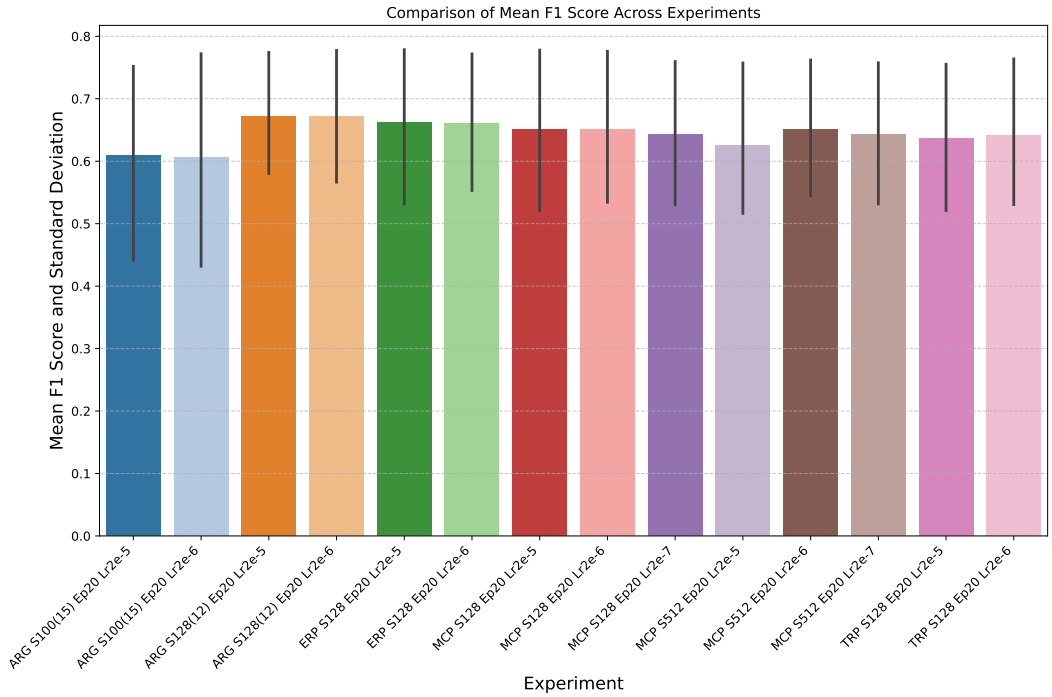


**Figure 6.3:** Mean *F1 Score* with standard deviation across all scenarios for model *st5-base* fine-tuned with our pipeline. Refer to Table 6.1 for the entire result set and abbreviations.

accounting for close to 0%, the training data offers very limited teaching potential. With easy non-matches vastly outnumbering meaningful pairs, the model struggles to learn meaningful distinctions between matches and non-matches, solely aligning with the target domain.

Surprisingly, *Equally Random Pairs (TRP)* sampling delivers the strongest results among pair-based strategies. Compared to the *Most Certain Pairs (MCP)* strategy, the random selection from weak label classes successfully creates a diverse training set that the model can effectively learn from. Interestingly, potentially unreliable weak labels around the decision boundary do not appear to negatively impact the model's performance.

*Most Certain Pairs (MCP)* sampling was further evaluated with additional hyper-parameters, particularly an increased sample size of 512, to compensate for the inherently less diverse training set. However, our results indicate that the additional pairs only exhibited a minor impact on mean matching performance. In fact, Table 6.1 suggests a contrary trend in some scenarios (e.g., D7), hinting at over-training (or over-fitting) as a potential risk to model performance (as discussed in Section 5.4).

Overall, our findings highlight the strong and consistent performance of *Anchored Record Groups (ARG)* fine-tuning across most scenarios. Notably, the influence of hyper-parameters, if within the GPU memory constraints, appears limited, attesting to the strategy's robust generalizability.

**(ii) Hyper-Parameters.** In contrast, the choice of hyper-parameters notably impacts the performance of pair-based strategies. As shown in Table 6.1, conflicting trends between learning rate, sample count, and scenario make identifying an ideal configuration inherently challenging. Overall, the initial learning rate between $2^{-5}$ and $2^{-6}$ exhibits only a minor impact on the matching performance, with one notable exception: sampling 512 pairs. For these experiments, decreasing the learning rate during training improved results. Despite these nuances, the default learning rate of $2^{-5}$ delivers the strongest results, particularly when paired with the *Anchored Record Groups (ARG)* strategy, which achieved the best experimental outcomes.

**(iii) Fine-tuning vs. Baselines.** The primary objective of fine-tuning is to enhance the matching performance of pre-trained embedding language models, particularly *st5-base*. Our results confirm a positive impact for most scenarios, especially for D5 and the challenging D10. However, some scenarios, such as D2 or D9, show a slight decline in matching performance with certain strategies. Comparing the best-performing strategy for each scenario against the baseline suggests that these variations are likely influenced by the weak label creation process itself. As discussed in Section 5.3, not all scenarios produce equally reliable

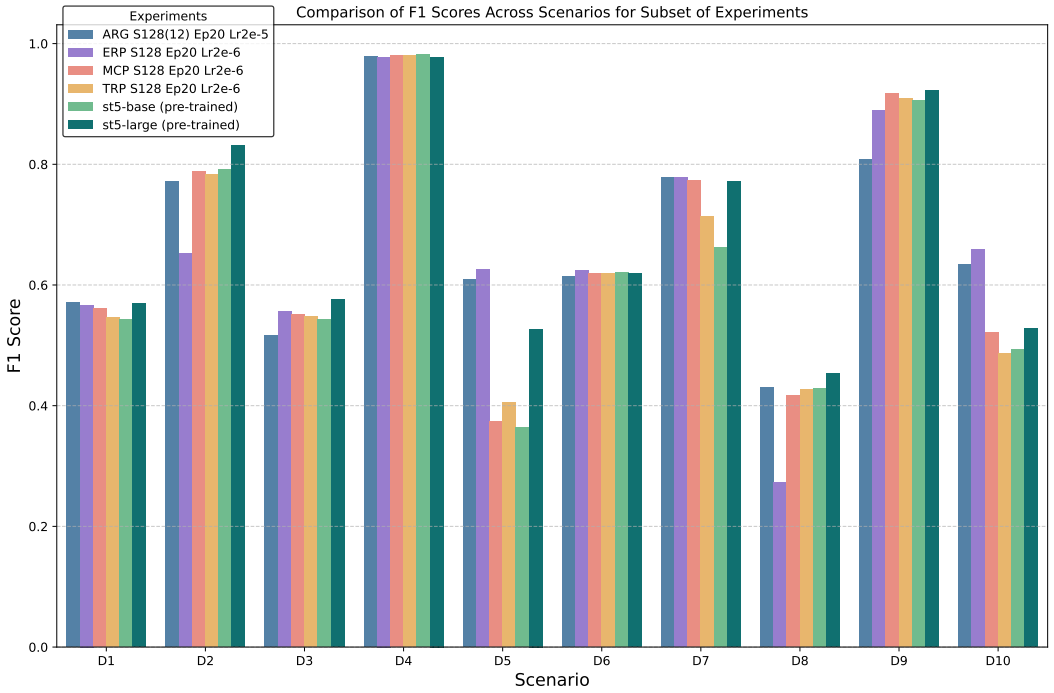


**Figure 6.4:** Mean *F1 score* for the strongest experiment per training strategy across all scenarios compared against the pre-trained models *st5-base* and *st5-large*. Refer to Table 6.1 for the entire result set.

weak labels (see Figure 5.5). Consequently, the knowledge available for fine-tuning is inherently limited, further constrained by the lack of meaningful content in some selected pairs.

Overall, the *Anchored Record Groups (ARG)* strategy with 128 samples, 12 negatives per anchor, 20 training epochs, and an initial learning rate of $2^{-5}$ delivers the strongest results, improving the mean *F1* performance of the base model by 0.043 – with some scenarios by up to 0.246 (D5).

**Transfer Learning Potential.** Interestingly, we observed that the matching performance for scenario D6 varies depending on whether the embedding model is fine-tuned with weak labels from D5, D6 or D7. For example, matching with a model trained on D5 yields an *F1 score* of 0.626, compared to 0615 when trained on D6 and 0.627 on D7 (fine-tuned with ARG, 128 (12) samples, 20 epochs and an initial learning rate of $2^{e-5}$). This variation suggests potential for transfer learning when fine-tuning embedding models, presumably due to the generic nature of pre-trained models.

## 6.4 The Last Mile: Matching Records

As part of our experiments, we assume the *F1* performance of each run as the optimal *F1 score* achieved across the entire set of possible thresholds, $\delta \in [0.05, 0.95]$ with a step size of 0.05, (in alignment with the methodology of Zeakis et al. as described in Section 4.1). While this approach is viable in research settings where ground truth annotations are available, it significantly limits the reproducibility of our results in practical applications. As motivated in Section 2.4, a truly unsupervised ER approach should be capable of automatically determining reasonable parameters – in this case, an appropriate threshold $\delta$.

The consequences of choosing a poor threshold can be significant, as illustrated in Figure 6.5. For the pre-trained models, only two threshold values, 0.50 and 0.55, produce acceptable results. In contrast, the fine-tuned model demonstrates greater resilience to the choice of threshold. A closer analysis reveals that its similarity values are more evenly distributed, resulting in a gradual increase in precision and a simultaneous decrease in recall as the threshold $\delta$ is raised. This effect is particularly significant for challenging scenarios, such as D10, indicating that the fine-tuned model exhibits more stable and robust matching abilities. Additionally, subsequent thresholding techniques, like those outlined in Section 5.2, similarly benefit from a more even distribution.

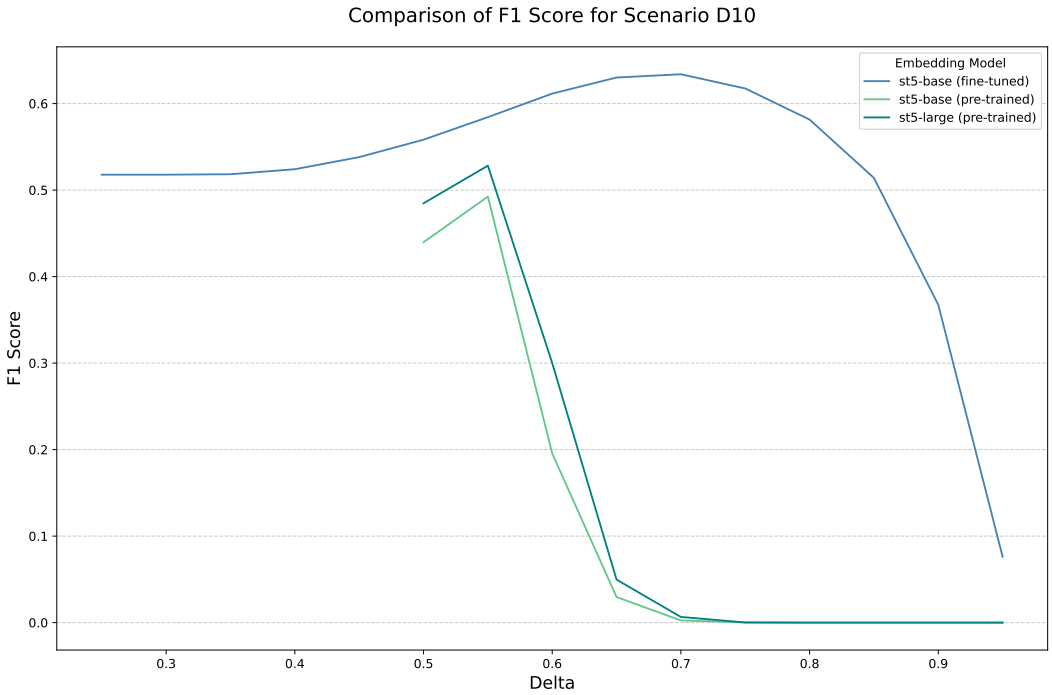


**Figure 6.5:** *F1 score* of pre-trained and fine-tuned embedding models for scenario D10 across various values of $\delta$. Model *st5-base* fine-tuned with ARG, 128 (12) samples, 20 epochs and an initial learning rate of $2^{e-5}$.

| Config | Samples | Ep. | Lr. | $\mathbf{D_1}$ | $\mathbf{D_2}$ | $\mathbf{D_3}$ | $\mathbf{D_4}$ | $\mathbf{D_5}$ | $\mathbf{D_6}$ | $\mathbf{D_7}$ | $\mathbf{D_8}$ | $\mathbf{D_9}$ | $\mathbf{D_{10}}$ | $\mu_D$ | $\sigma_D$ |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| *BASE* | – | – | – | 0.543 | 0.791 | 0.543 | 0.982 | 0.364 | 0.621 | 0.662 | 0.428 | 0.906 | 0.492 | 0.633 | 0.203 |
| *LARGE* | – | – | – | 0.570 | 0.832 | 0.576 | 0.978 | 0.527 | 0.619 | 0.771 | 0.453 | 0.923 | 0.528 | 0.678 | 0.184 |
| ARG | 100 (15) | 20 | $2^{-5}$ | 0.576 | 0.740 | 0.540 | 0.976 | 0.582 | – | 0.769 | 0.432 | 0.841 | 0.630 | 0.676 | 0.169 |
| ARG | 100 (15) | 20 | $2^{-6}$ | 0.559 | 0.790 | 0.549 | 0.979 | 0.482 | – | 0.789 | 0.439 | 0.932 | 0.543 | 0.673 | 0.201 |
| ARG | 128 (12) | 20 | $2^{-5}$ | 0.571 | 0.772 | 0.516 | 0.978 | 0.610 | 0.615 | 0.778 | 0.430 | 0.809 | 0.634 | 0.671 | 0.162 |
| ARG | 128 (12) | 20 | $2^{-6}$ | 0.559 | 0.790 | 0.555 | 0.979 | 0.494 | 0.614 | 0.788 | 0.446 | 0.933 | 0.555 | 0.671 | 0.187 |
| ERP | 128 | 20 | $2^{-5}$ | 0.584 | 0.628 | 0.548 | 0.983 | 0.629 | 0.617 | 0.816 | 0.245 | 0.910 | 0.658 | 0.662 | 0.207 |
| ERP | 128 | 20 | $2^{-6}$ | 0.567 | 0.652 | 0.556 | 0.978 | 0.626 | 0.624 | 0.779 | 0.272 | 0.890 | 0.659 | 0.660 | 0.194 |
| MCP | 128 | 20 | $2^{-5}$ | 0.575 | 0.802 | 0.552 | 0.972 | 0.337 | 0.617 | 0.778 | 0.398 | 0.921 | 0.551 | 0.650 | 0.212 |
| MCP | 128 | 20 | $2^{-6}$ | 0.561 | 0.789 | 0.552 | 0.980 | 0.374 | 0.619 | 0.773 | 0.417 | 0.917 | 0.522 | 0.650 | 0.206 |
| MCP | 128 | 20 | $2^{-7}$ | 0.545 | 0.792 | 0.546 | 0.979 | 0.390 | 0.614 | 0.740 | 0.421 | 0.912 | 0.497 | 0.644 | 0.203 |
| MCP | 512 | 20 | $2^{-5}$ | 0.584 | 0.768 | 0.525 | 0.975 | 0.448 | 0.621 | 0.460 | 0.379 | 0.928 | 0.568 | 0.626 | 0.202 |
| MCP | 512 | 20 | $2^{-6}$ | 0.588 | 0.772 | 0.552 | 0.980 | 0.420 | 0.618 | 0.707 | 0.402 | 0.925 | 0.550 | 0.651 | 0.195 |
| MCP | 512 | 20 | $2^{-7}$ | 0.549 | 0.787 | 0.548 | 0.978 | 0.397 | 0.612 | 0.723 | 0.419 | 0.915 | 0.507 | 0.644 | 0.200 |
| TRP | 128 | 20 | $2^{-5}$ | 0.545 | 0.723 | 0.548 | 0.982 | 0.405 | 0.619 | 0.714 | 0.427 | 0.909 | 0.486 | 0.636 | 0.195 |
| TRP | 128 | 20 | $2^{-6}$ | 0.545 | 0.783 | 0.548 | 0.981 | 0.405 | 0.619 | 0.714 | 0.427 | 0.909 | 0.486 | 0.642 | 0.199 |

**Table 6.1:** Matching performance of the fine-tuned *st5-base* model, reported as *F1 scores*. For each scenario, weak labels are generated, and the model is fine-tuned using the specified number of samples, epochs, and learning rate. All random seeds are set to 42. Sample counts for the *ARG* strategy are formatted as "|anchors| (|negatives|)" and failed runs indicated by '–'. *Abbreviations: Training Epochs (Ep), Learning Rate (Lr).*

# 7 To Label or Not to Label: The Conclusion of This Work

At the beginning of this thesis, we set out to develop an entirely unsupervised pipeline capable of delivering robust performance across diverse entity resolution scenarios. Specifically, our focus was on fine-tuning embedding language models using automatically generated weak labels. In Section 6.3, our evaluation highlighted the strong yet scenario-dependent impact of our pipeline on matching performance. These findings are particularly rewarding, as many of the concepts introduced in this thesis represent foundational steps into new research areas and hold significant potential for future advancements.

The foundation of our pipeline lies in the weak label creation process. As concluded in Section 5.3, the performance of our weak labels is remarkable given their unsupervised nature. However, their effectiveness varies across the scenarios provided by Zeakis et al. [Zea+23a], revealing a lack of generality and robustness in some scenarios. Presumably unseen matching scenarios, such as "Markt-Pilot" introduced by Rettenmeier and Jesser in 2023 [RJ23], could provide further insights into the real-world performance. This behavior is unsurprising, as the task of creating weak labels is inherently similar to unsupervised entity resolution itself.

**Similarity Features.** Similar to rule-based ER approaches, our pipeline utilizes similarity measures to extract features from attributes shared across both data sources. However, a notable limitation of this approach is its dependency on identical attribute names and concepts. For example, in scenario D10, we generate features solely based on the "title" attribute, leaving other overlapping values, such as "actor name" (DBP) and "starring" ($\text{IMDb}_2$) untapped. While manual schema alignment represents one possible solution to this challenge, automating the process through an initial schema matching stage [RB01] presents the preferred option.

Another challenge lies in selecting the proper similarity measure for a given attribute. As discussed in Section 5.1, we distinguish solely between textual and numeric values to maintain generality and reduce complexity. To further enhance the quality of generated similarity features, future research could explore additional attribute categories, such as long versus short textual values or categorical data, and evaluate tailored similarity measures for each. Automated similarity measure detection, proposed by Loster et al. [LKN21], presents an innovative solution to this challenge. By incorporating diverse similarity measures and automated detection

methods, this stage of our pipeline could be further refined to increase practical applicability and robustness.

**Similarity Thresholds.** Mapping continuous similarity features to binary labels required additional foundational work on automated threshold selection. Surprisingly, limited prior research existed in this area, limiting potential foundational concepts to build upon. As discussed in Section 5.2, our results suggest that thresholds can indeed be selected automatically; however, their effectiveness closely relates to the discriminative power of the underlying similarity feature. Appendix A illustrates significant variations in the quality of the features, which renders selecting an optimal thresholding technique solely on our evaluation results challenging. This underscores the need for further research in threshold selection, ideally tied to advancements in the creation of similarity features.

**Label Aggregation.** Unlike the thresholding stage, label aggregation has been studied extensively, particularly in the areas of crowdsourcing and programmatic weak supervision (PWS). In Section 5.3, we compared several existing approaches and adapted a data fusion algorithm to our use-case. Surprisingly, majority voting – a notably simple approach – showed the most consistent results across various scenarios. Investigating the reasons behind this discrepancy offers an opportunity for future research work and can could enhance the quality of weak labels – and subsequently the performance of the entire pipeline.

**Fine-Tuning Process.** Chapter 6 focused extensively on the fine-tuning process, evaluating training objectives and hyper-parameter settings. Building on the strong foundational performance from the STS task, our goal was to adapt the pre-trained embedding model to the target domain of each scenario, enhancing its ability to distinguish between matches and non-matches. Among the objectives tested, training with grouped records produced the most significant training signals, consistent with the findings of Chang et al. [CAG24].

Over-optimizing the training process to specific scenarios negatively affects the generalizability of the entire pipeline. To mitigate this, we focused on evaluating a concise set of configurations and observed that the hyper-parameter values had only a minor influence on the matching performance. Notably, our results suggest that hyper-parameters can be optimized without reliance on ground truth annotations, instead leveraging metrics such as the loss value. Subsequent work could introduce an unsupervised optimization framework to facilitate a smooth and scenario-specific fine-tuning process.

Another promising direction lies in transfer learning to enhance the effectiveness of training with weak labels. Our experiments demonstrated improved matching performance when a model fine-tuned on other movie scenarios (such as D5 or D7) was applied to scenario D6. This suggests that pre-trained embedding models

benefit from transfer learning, particularly when weak labels in a given scenario are unreliable.

**Evaluation Metrics.** A notable limitation of the presented pipeline is the absence of additional "unsupervised" evaluation metrics for its individual stages (i.e., independent of a ground truth annotation). Such metrics could include inherent algorithmic measures, such as the correlation score achieved while thresholding with Pearson correlation, or unsupervised matching scores that assess the matching quality. Including these metrics would enable more data-driven decisions at each stage of the pipeline, helping to mitigate downstream effects caused by suboptimal choices in earlier stages.

**Final Remarks.** The core question that motivated this thesis remains: *Does our pipeline advance unsupervised entity resolution towards a viable option in practice?*

As defined in Section 2.4, our pipeline operates entirely unsupervised. With a maximum runtime under two hours for its strongest configuration, it demonstrates practical applicability, particularly in comparison to the high runtimes of ZeroER [Wu+20; Zea+23b]. Remarkably, this performance is achieved despite the framework being research-driven rather than production-optimized.

We view this thesis as a starting point, offering numerous opportunities for future research. Specifically, we encourage fellow researchers to challenge our design decision and address limitations, broadening the pipeline's applicability – for instance, by adapting it to *Dirty ER*. For our pipeline to become a viable option to data cleaning efforts in practice, we envision our concepts integrated into user-friendly ER frameworks, such as *pyJedAI* or *RecordLinkage*, that provide a guided workflow as well as options for complementary tasks.

In conclusion, this thesis advances the state of unsupervised entity resolution while also laying groundwork for progress in related areas, such as automated thresholding. We are eager to see how future work will build upon these contributions.

# A Weak Label Generation: Detailed Evaluation

## A.1 Feature Creation: The Distinguishing Ability of Similarity Measures

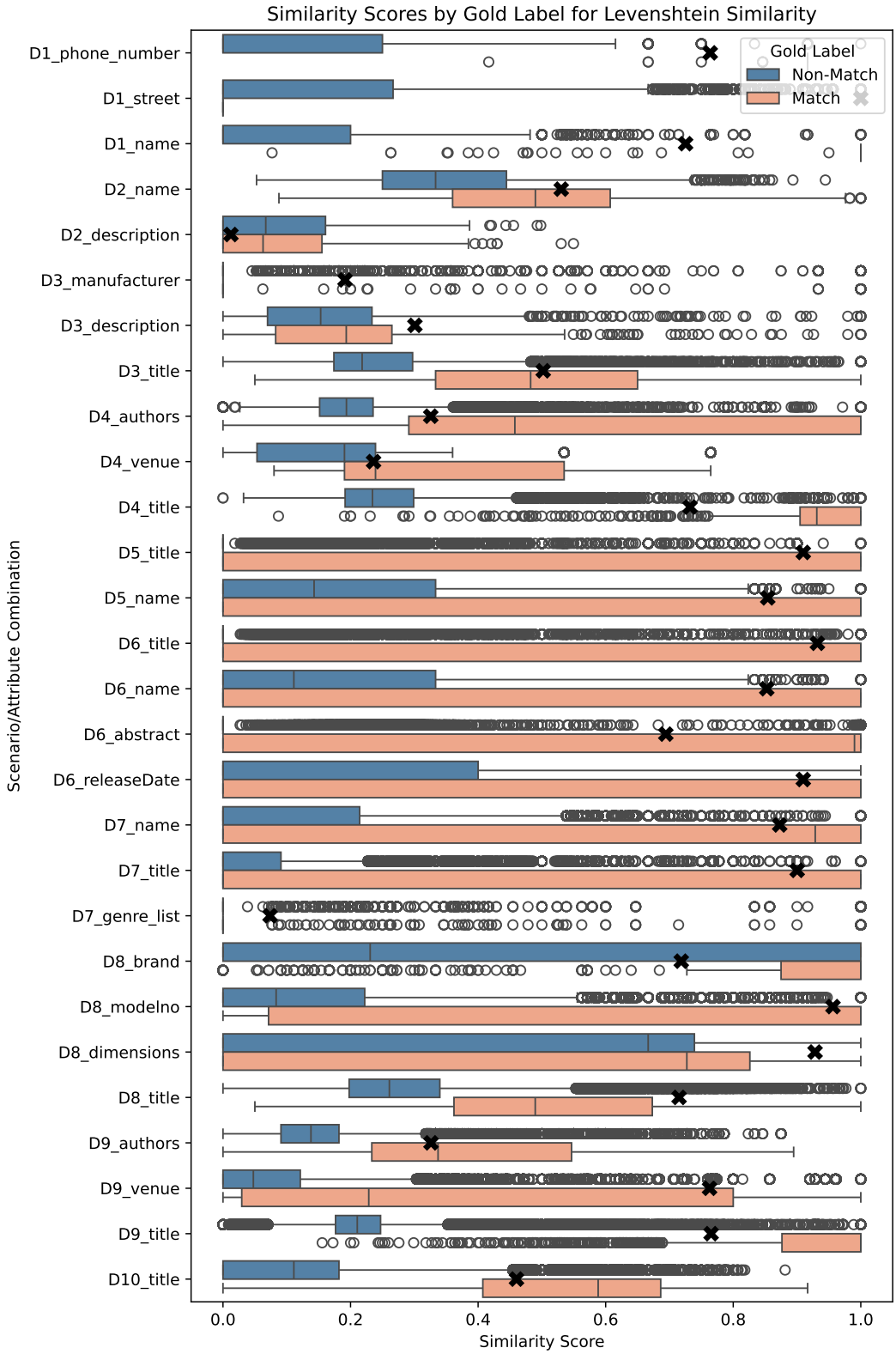


**Figure A.1:** Distribution of labels according to the similarity measure "Levenshtein" split by their ground truth label. The optimal similarity threshold is marked with an X.

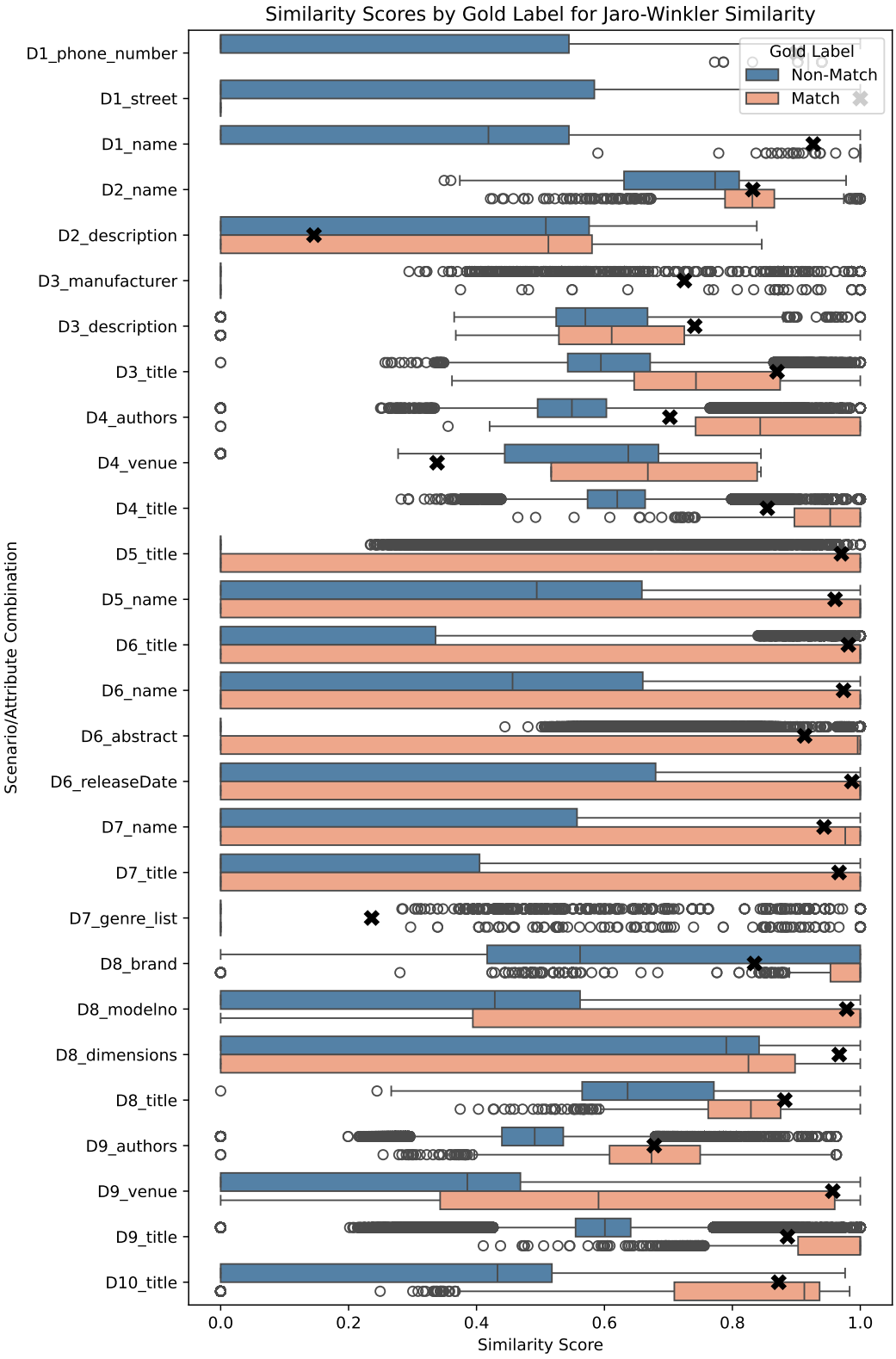


**Figure A.2:** Distribution of labels according to the similarity measure "Jaro-Winkler" split by their ground truth label. The optimal similarity threshold is marked with an X.

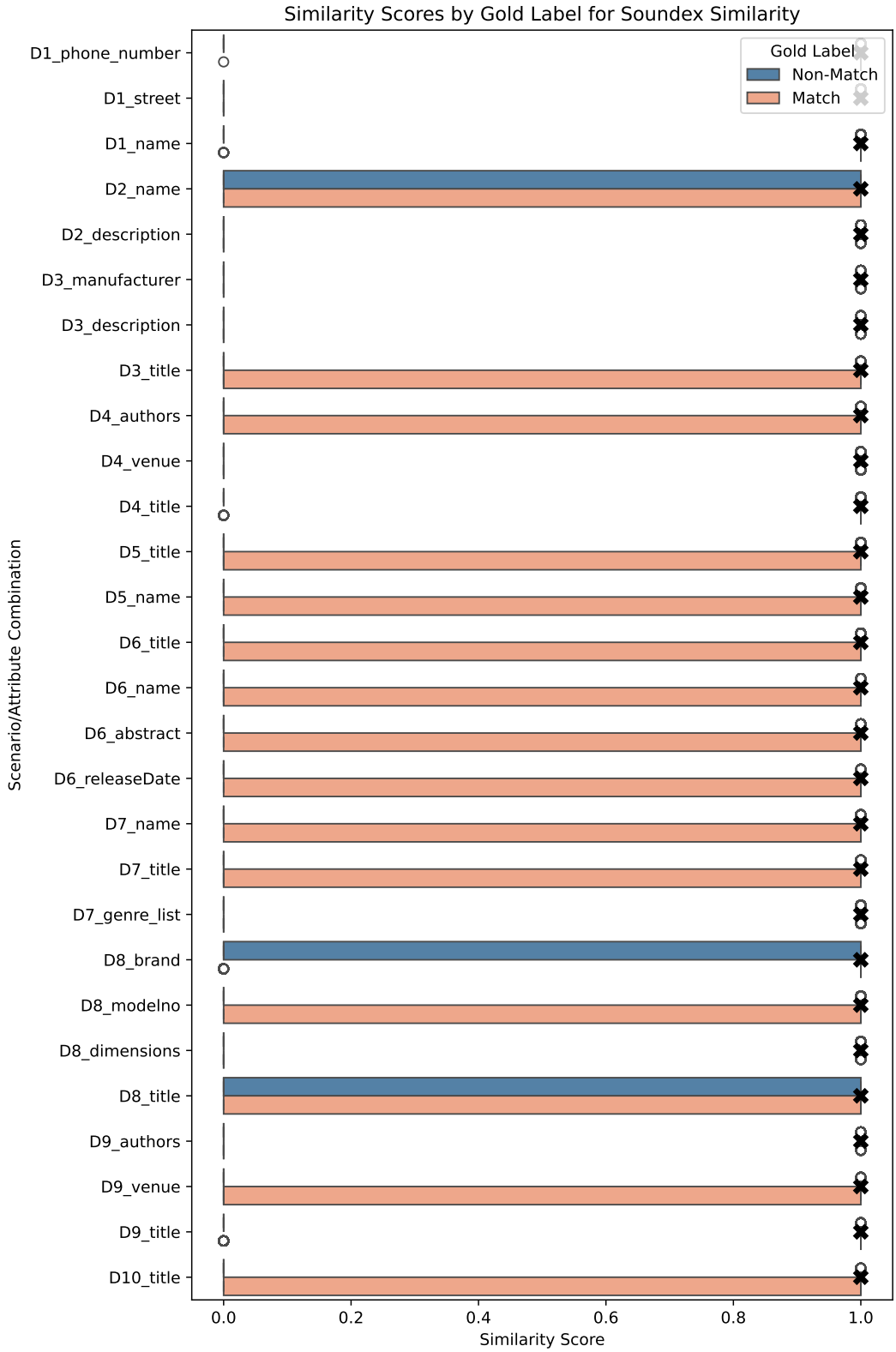


**Figure A.3:** Distribution of labels according to the similarity measure "Soundex" split by their ground truth label. The optimal similarity threshold is marked with an X.

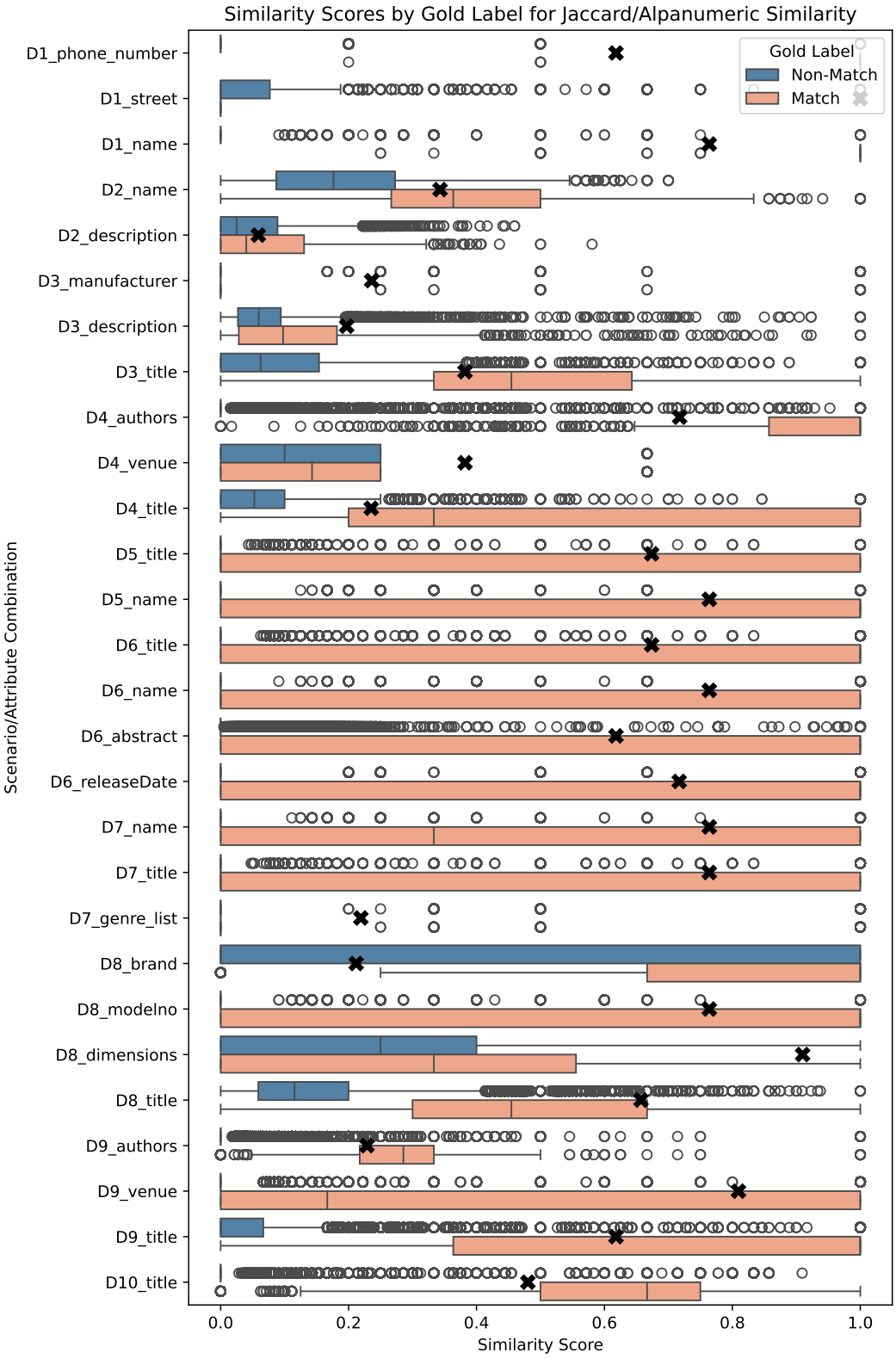


**Figure A.4:** Distribution of labels according to the similarity measure "Jaccard/Alphanumeric" split by their ground truth label. The optimal similarity threshold is marked with an X.

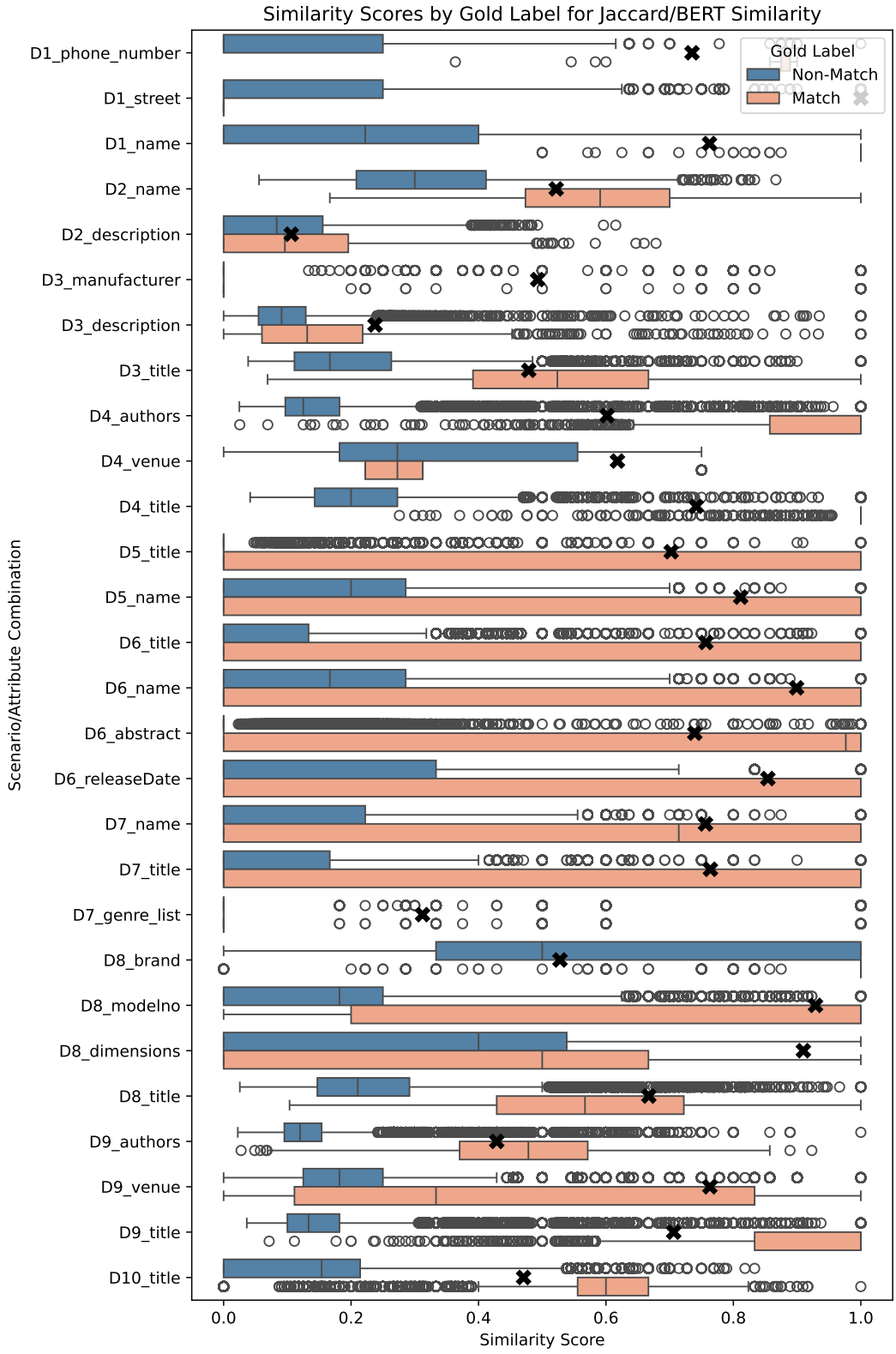


**Figure A.5:** Distribution of labels according to the similarity measure "Jaccard/BERT" split by their ground truth label. The optimal similarity threshold is marked with an X.

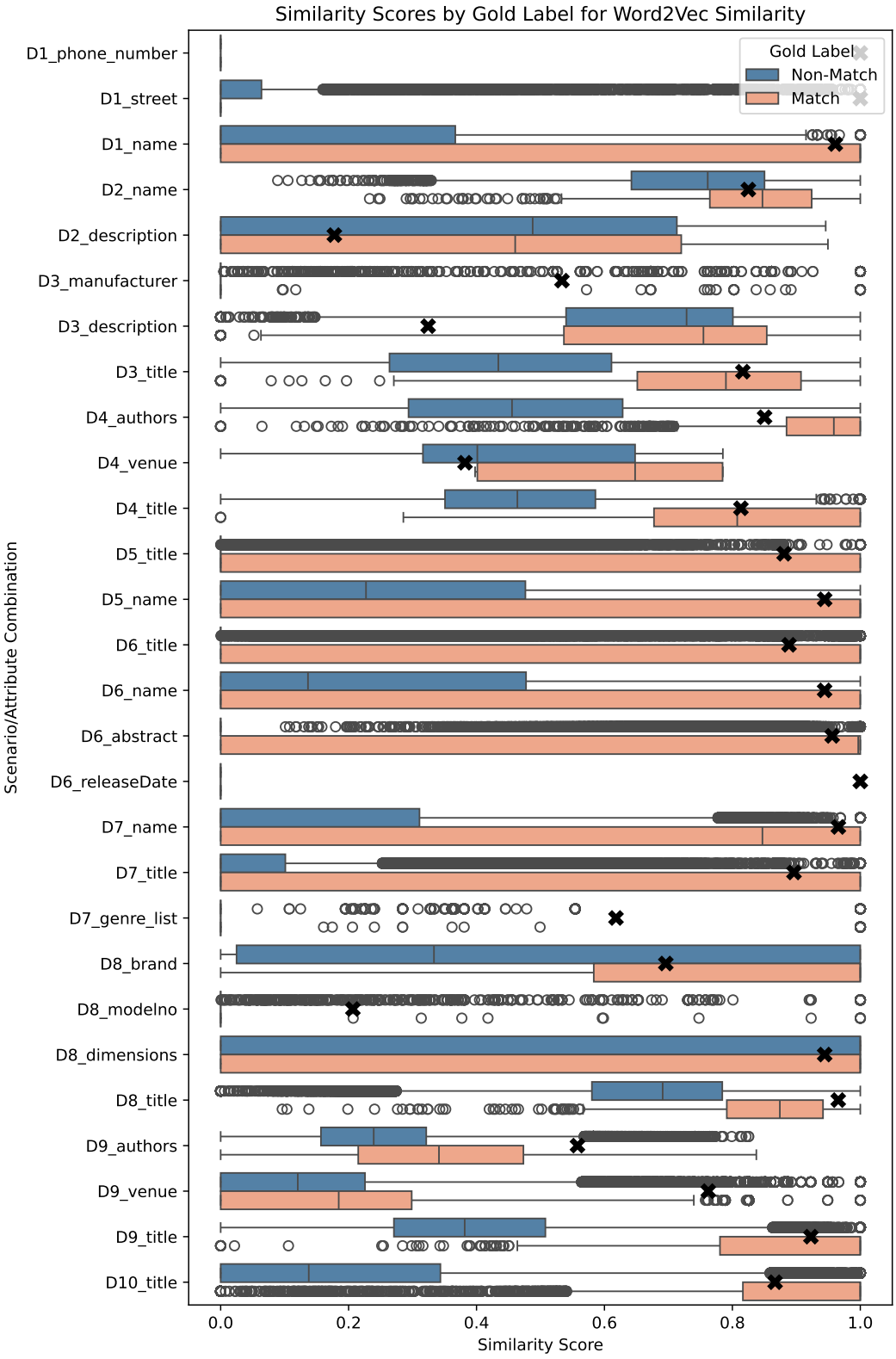


**Figure A.6:** Distribution of labels according to the similarity measure "Word2Vec" split by their ground truth label. The optimal similarity threshold is marked with an X.

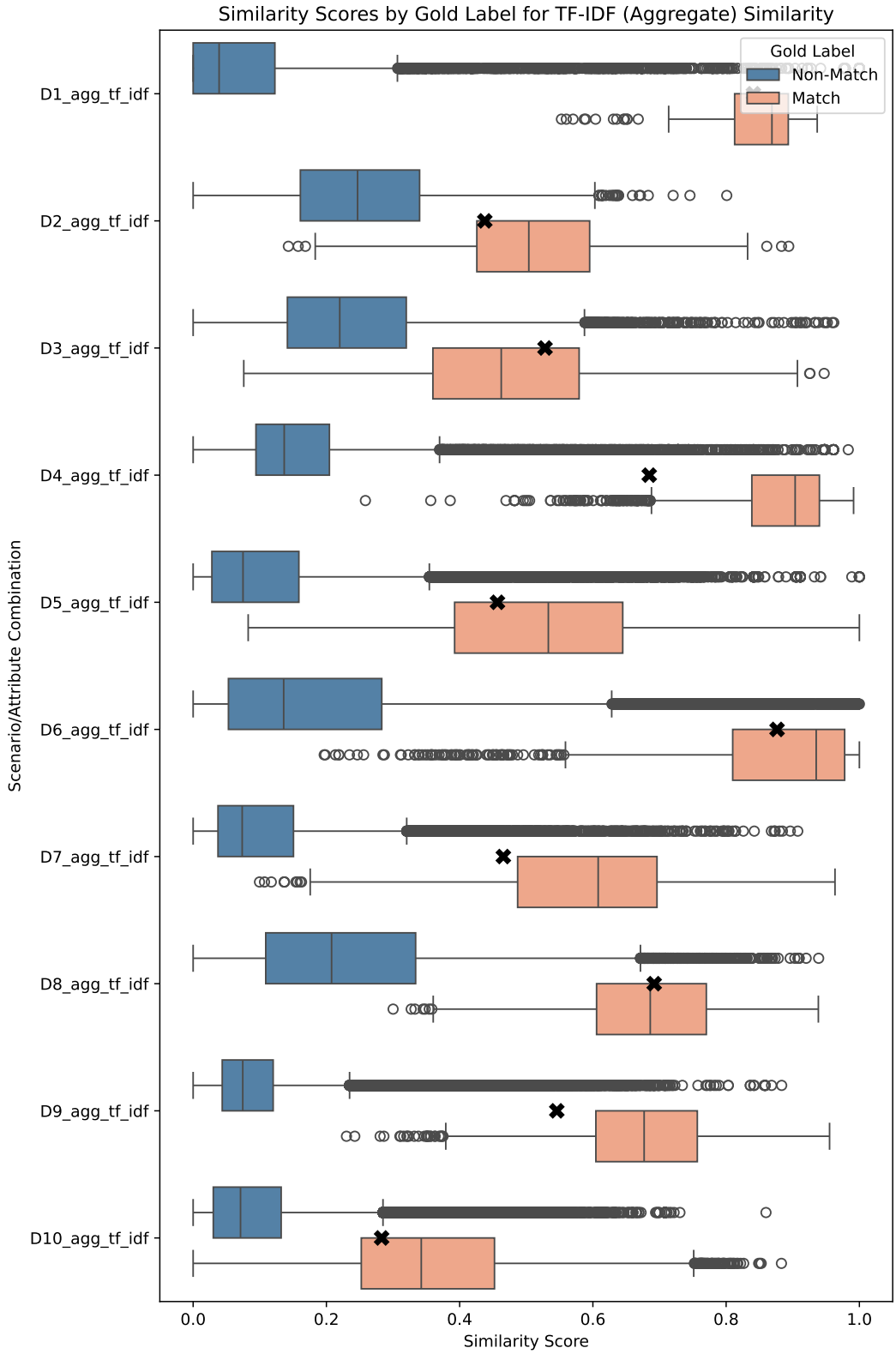


**Figure A.7:** Distribution of labels according to the similarity measure "TF-IDF" split by their ground truth label. The optimal similarity threshold is marked with an X.

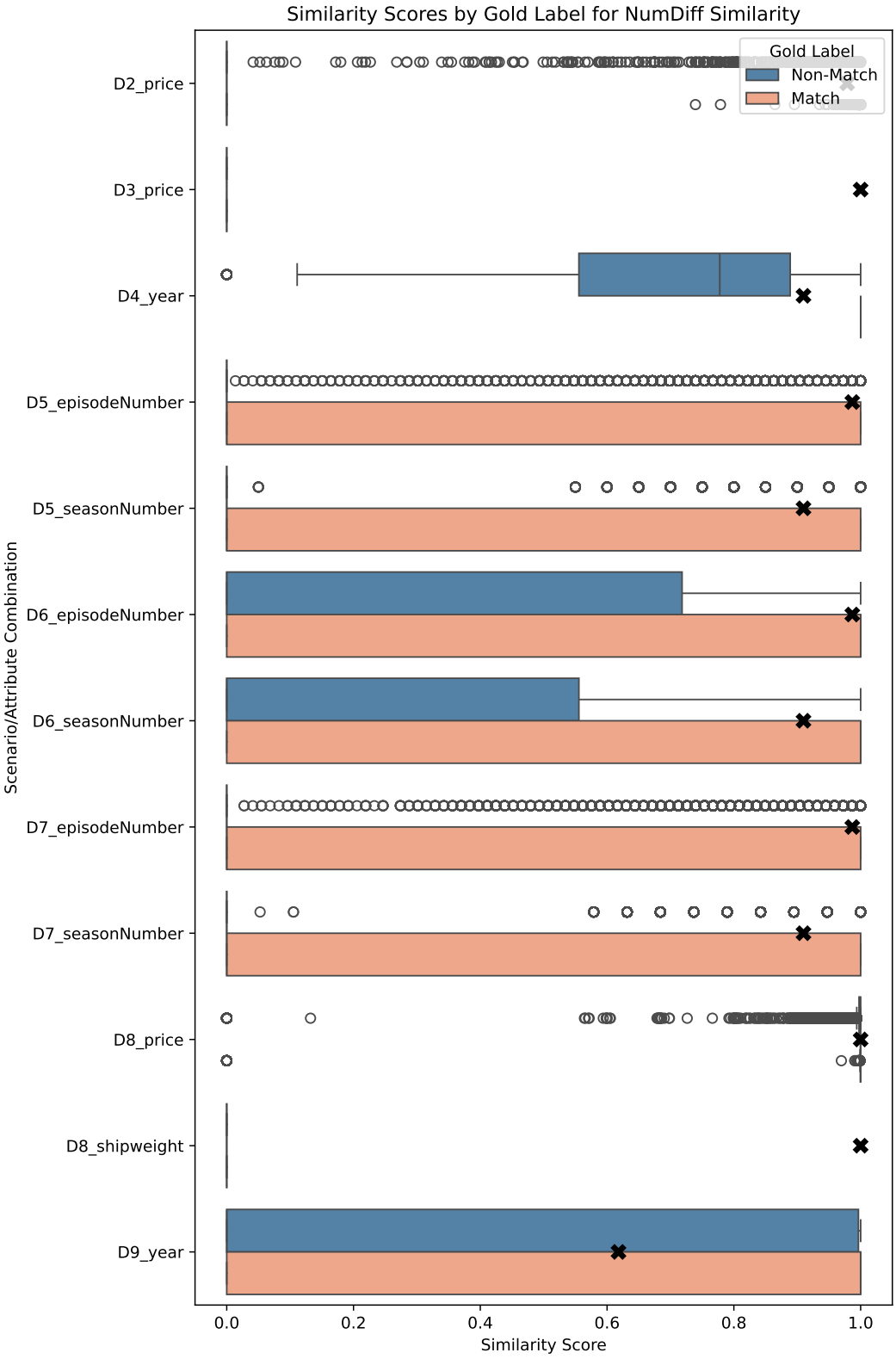


**Figure A.8:** Distribution of labels according to the similarity measure "NumDiff" split by their ground truth label. The optimal similarity threshold is marked with an X.

# Bibliography


[AA] The U.S. National Archives and Records Administration. *The Soundex Indexing System*. Accessed: 01.01.2025. National Archives. URL: https://www.archives.gov/research/census/soundex (see page 38).

[Ahl19] Rex Ahlstrom. **The Role Of Data In The Age Of Digital Transformation**. *Forbes* (2019). Accessed: 30.12.2024. URL: https://www.forbes.com/councils/forbestechcouncil/2019/01/17/the-role-of-data-in-the-age-of-digital-transformation/ (see page 1).

[BG06] Indrajit Bhattacharya and Lise Getoor. **A Latent Dirichlet Model for Unsupervised Entity Resolution**. In: *Proceedings of the Sixth SIAM International Conference on Data Mining*. 2006, 47–58 (see page 13).

[BG21] Nils Barlaug and Jon Atle Gulla. **Neural Networks for Entity Matching: A Survey**. *ACM Transactions on Knowledge Discovery from Data (TKDD)* 15:3 (2021), 52:1–52:37 (see page 12).

[CAG24] Johnny Chang, Kaushal Alate, and Kanu Grover. **"That was smooth": Exploration of S-BERT with Multiple Negatives Ranking Loss and Smoothness-Inducing Regularization**. Seminar Paper. Accessed: 27.12.2024. Stanford University, 2024. URL: https://web.stanford.edu/class/archive/cs/cs224n/cs224n.1244/final-projects/JohnnyChangKanuGroverKaushalAtulAlate.pdf (see pages 64, 78).

[CCR20] Victor Christen, Peter Christen, and Erhard Rahm. **Informativeness-Based Active Learning for Entity Resolution**. In: *European Conference on Principles of Data Mining and Knowledge Discovery (PKDD)*. 2020, 125–141 (see pages 13, 58).

[Che+23] Hao Chen, Yiming Zhang, Qi Zhang, Hantao Yang, Xiaomeng Hu, Xuetao Ma, Yifan Yanggong, and Junbo Zhao. **Maybe Only 0.5% Data is Needed: A Preliminary Exploration of Low Training Data Instruction Tuning** (2023). arXiv: 2305.09246 (see page 55).

[Chr12] Peter Christen. **Data Matching**. Berlin – Heidelberg – New York: Springer Verlag, 2012 (see pages 2, 9, 11, 18, 35, 38).

[DBS13] Xin Luna Dong, Laure Berti-Équille, and Divesh Srivastava. "Data Fusion: Resolving Conflicts from Multiple Sources." In: *Handbook of Data Quality, Research and Practice*. Ed. by Shazia W. Sadiq. Springer, 2013, 293–318 (see page 51).

[Dev+19] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. **BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding**. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. 2019, 4171–4186 (see pages 12, 15, 39).

[DHI12] AnHai Doan, Alon Y. Halevy, and Zachary G. Ives. **Principles of Data Integration**. Morgan Kaufmann, 2012 (see pages 35, 37).

[DN13] Uwe Draisbach and Felix Naumann. **On Choosing Thresholds for Duplicate Detection**. In: *Proceedings of the International Conference on Information Quality (ICIQ)*. 2013 (see page 43).

[DS79] Alexander Philip Dawid and Allan M Skene. **Maximum Likelihood Estimation of Observer Error-Rates Using the EM Algorithm**. *Journal of the Royal Statistical Society: Series C (Applied Statistics)* 28:1 (1979), 20–28 (see page 50).

[Ebr+18] Muhammad Ebraheem, Saravanan Thirumuruganathan, Shafiq Joty, Mourad Ouzzani, and Nan Tang. **Distributed representations of tuples for entity resolution**. *PVLDB* 11:11 (2018), 1454–1467. ISSN: 2150-8097 (see page 12).

[Eft+23] Vasilis Efthymiou, Ekaterini Ioannou, Manos Karvounis, Manolis Koubarakis, Jakub Maciejewski, Konstantinos Nikoletos, George Papadakis, Dimitrios Skoutas, Yannis Velegrakis, and Alexandros Zeakis. **Self-configured Entity Resolution with pyJedAI**. In: *IEEE International Conference on Big Data*. 2023, 339–343 (see page 12).

[Gra+22] Martin Graf, Lukas Laskowski, Florian Papsdorf, Florian Sold, Roland Gremmelspacher, Felix Naumann, and Fabian Panse. **Frost: A Platform for Benchmarking and Exploring Data Matching Results**. *PVLDB* 15:12 (2022), 3292–3305 (see page 12).

[Hen+17] Matthew Henderson, Rami Al-Rfou, Brian Strope, Yun-Hsuan Sung, László Lukács, Ruiqi Guo, Sanjiv Kumar, Balint Miklos, and Ray Kurzweil. **Efficient Natural Language Response Suggestion for Smart Reply** (2017). arXiv: 1705.00652 (see page 64).

[HN20] Mazhar Hameed and Felix Naumann. **Data Preparation: A Survey of Commercial Tools**. *SIGMOD Record* 49:3 (Sept. 2020) (see page 1).

[Jar89] Matthew A Jaro. **Advances in Record-Linkage Methodology as Applied to Matching the 1985 Census of Tampa, Florida**. *Journal of the American Statistical Association* 84:406 (1989), 414–420 (see page 37).

[Kon+16] Pradap Konda, Sanjib Das, Paul Suganthan G. C., AnHai Doan, Adel Ardalan, Jeffrey R. Ballard, Han Li, Fatemah Panahi, Haojun Zhang, Jeffrey F. Naughton, Shishir Prasad, Ganesh Krishnan, Rohit Deep, and Vijay Raghavendra. **Magellan: Toward Building Entity Matching Management Systems**. In: *PVLDB*. Vol. 9. 12. 2016, 1197–1208 (see pages 12, 37).

[KTR10] Hanna Köpcke, Andreas Thor, and Erhard Rahm. **Evaluation of entity resolution approaches on real-world match problems**. *PVLDB* 3:1 (2010), 484–493 (see pages 29, 43).

[Las23] Lukas Laskowski. **NumbER – Entity Resolution on Numerical Data**. Master's Thesis. Potsdam, Germany: Hasso-Plattner-Institut, Digital Engineering Faculty at the University of Potsdam, 2023 (see page 41).

[Li+20] Yuliang Li, Jinfeng Li, Yoshihiko Suhara, AnHai Doan, and Wang-Chiew Tan. **Deep Entity Matching with Pre-Trained Language Models**. *PVLDB* 14:1 (2020), 50–60. DOI: 10.14778/3421424.3421431. URL: http://www.vldb.org/pvldb/vol14/p50-li.pdf (see pages 2, 12, 14, 29).

[Li+24] Huahang Li, Shuangyin Li, Fei Hao, Chen Jason Zhang, Yuanfeng Song, and Lei Chen. **BoostER: Leveraging Large Language Models for Enhancing Entity Resolution**. In: *Companion Proceedings of the ACM on Web Conference 2024*. 2024, 1043–1046 (see page 13).

[Li22] Hang Li. **Language Models: Past, Present, and Future**. *Communications of the ACM* 65:7 (2022), 56–63 (see page 15).

[LKN21] Michael Loster, Ioannis Koumarelas, and Felix Naumann. **Knowledge Transfer for Entity Resolution with Siamese Neural Networks**. *Journal on Data and Information Quality* 13:1 (2021), 2:1–2:25 (see page 77).

[LL24] Xianming Li and Jing Li. **AoE: Angle-optimized Embeddings for Semantic Textual Similarity**. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2024, 1825–1839 (see page 62).

[LS23] Lukas Laskowski and Florian Sold. **Explainable Data Matching: Selecting Representative Pairs with Active Learning Pair-Selection Strategies**. In: *Proceedings of the Conference Datenbanksysteme in Business, Technologie und Web Technik (BTW)*. 2023, 1099–1104 (see page 58).

[McK24] QuantumBlack AI by McKinsey. *The state of AI in early 2024: Gen AI adoption spikes and starts to generate value*. Accessed: 30.12.2024. 2024. URL: https://www.mckinsey.com/capabilities/quantumblack/our-insights/the-state-of-ai#/ (see page 1).

[Mik+13] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg Corrado, and Jeffrey Dean. **Distributed Representations of Words and Phrases and their Compositionality**. *Advances in Neural Information Processing Systems* 26 (2013) (see pages 12, 15, 39).

[Mud+18] Sidharth Mudgal, Han Li, Theodoros Rekatsinas, AnHai Doan, Youngchoon Park, Ganesh Krishnan, Rohit Deep, Esteban Arcaute, and Vijay Raghavendra. **Deep Learning for Entity Matching: A Design Space Exploration**. In: *Proceedings of the International Conference on Management of Data (SIGMOD)*. 2018, 19–34 (see page 29).

[NH10] Felix Naumann and Melanie Herschel. **An Introduction to Duplicate Detection**. Morgan & Claypool, 2010 (see pages 9, 11).

[Ni+22] Jianmo Ni, Gustavo Hernández Ábrego, Noah Constant, Ji Ma, Keith B. Hall, Daniel Cer, and Yinfei Yang. **Sentence-T5: Scalable Sentence Encoders from Pre-trained Text-to-Text Models**. In: *Findings of the Association for Computational Linguistics: ACL*. 2022, 1864–1874 (see page 16).

[Ope24] OpenAI. *GPT-4 Technical Report*. 2024. arXiv: 2303.08774 (see pages 13, 16).

[OSR21] Daniel Obraczka, Jonathan Schuchart, and Erhard Rahm. **Embedding-Assisted Entity Resolution for Knowledge Graphs**. In: *Proceedings of the 2nd International Workshop on Knowledge Graph Construction co-located with 18th Extended Semantic Web Conference (ESWC 2021)*. 2021 (see page 29).

[Pap+11] George Papadakis, Ekaterini Ioannou, Claudia Niederée, and Peter Fankhauser. **Efficient Entity Resolution for Large Heterogeneous Information Spaces**. In: *Proceedings of the ACM International Conference on Web Search and Data Mining (WSDM)*. 2011, 535–544 (see page 30).

[Pap+21] George Papadakis, Ekaterini Ioannou, Emanouil Thanos, and Themis Palpanas. **The Four Generations of Entity Resolution**. Synthesis Lectures on Data Management. Morgan & Claypool Publishers, 2021 (see page 11).

[Pap+22] George Papadakis, Vasilis Efthymiou, Emmanouil Thanos, and Oktie Hassanzadeh. **Bipartite Graph Matching Algorithms for Clean-Clean Entity Resolution: An Empirical Evaluation**. In: *Proceedings of the International Conference on Extending Database Technology (EDBT)*. 2022, 2:462–2:474 (see page 27).

[Pap+24] George Papadakis, Nishadi Kirielle, Peter Christen, and Themis Palpanas. **A Critical Re-evaluation of Record Linkage Bench- marks for Learning-Based Matching Algorithms**. In: *Proceedings of the International Conference on Data Engineering (ICDE)*. 2024, 3435–3448 (see pages 30, 31).

[Pat22] Nick Patience. **Firms Realize the Value of Data Driven Decision Making**. *S&P Global Blog* (2022). Accessed: 30.12.2024. URL: https://www.spglobal.com/market-intelligence/en/news-insights/research/firms-realize-the-value-of-data-driven-decision-making (see page 1).

[PB20] Anna Primpeli and Christian Bizer. **Profiling Entity Matching Benchmark Tasks**. In: *Proceedings of the International Conference on Information and Knowledge Management (CIKM)*. 2020, 3101–3108 (see pages 30, 36, 41).

[PG95] Karl Pearson and Francis Galton. **VII. Note on Regression and Inheritance in the Case of Two Parents**. *Proceedings of the Royal Society of London* 58:347-352 (1895), 240–242 (see page 47).

[PGD23] Derek Paulsen, Yash Govind, and AnHai Doan. **Sparkly: A Simple yet Surprisingly Strong TF/IDF Blocker for Entity Matching**. *PVLDB* 16:6 (2023), 1507–1519 (see pages 11, 25).

[Phi90] Lawrence Philips. **Hanging on the Metaphone**. *Computer Language* 7:12 (1990) (see page 38).

[PSB25] Ralph Peeters, Aaron Steiner, and Christian Bizer. **Entity Matching using Large Language Models**. In: *Proceedings of the International Conference on Extending Database Technology (EDBT)*. 2025, 529–541 (see page 13).

[Raf+20] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. **Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer**. *Journal of Machine Learning Research* 21 (2020), 140:1–140:67 (see page 16).

[Rat+17] Alexander Ratner, Stephen H. Bach, Henry R. Ehrenberg, Jason Alan Fries, Sen Wu, and Christopher Ré. **Snorkel: Rapid Training Data Creation with Weak Supervision**. In: vol. 11. 3. 2017, 269–282 (see pages 18, 52).

[RB01] Erhard Rahm and Philip A. Bernstein. **A survey of approaches to automatic schema matching**. *VLDB Journal* 10(4) (2001), 334–350 (see pages 10, 77).

[RD23] Simon Ray and DCI. *WHITEPAPER: The hidden cost of bad data. How much is bad data costing your firm?* Accessed: 30.12.2024. 2023. URL: https://www.wealth-dci.com/wp-content/uploads/2023/01/dci-whitepaper-the_hidden_cost_of_bad_data.pdf (see page 1).

[RG19] Nils Reimers and Iryna Gurevych. **Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks**. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, EMNLP-IJCNLP*. 2019, 3980–3990 (see page 16).

[RJ23] Tobias Rettenmeier and Alexander Jesser. **Introducing a novel dataset for product matching: A new challenge for matching systems**. In: *5th International Communication Engineering and Cloud Computing Conference (CECCC)*. 2023, 26–29 (see page 77).

[San+11] Juliana B dos Santos, Carlos A Heuser, Viviane P Moreira, and Leandro K Wives. **Automatic Threshold Estimation for Data Matching Applications**. *Information Sciences* 181:13 (2011), 2685–2699 (see page 43).

[Sat+11] Ville Satopaa, Jeannie Albrecht, David Irwin, and Barath Raghavan. **Finding a "Kneedle" in a Haystack: Detecting Knee Points in System Behavior**. In: *31st International Conference on Distributed Computing Systems Workshops*. 2011, 166–171 (see page 45).

[SB88] Gerard Salton and Christopher Buckley. **Term-Weighting Approaches in Automatic Text Retrieval**. *Information Processing & Management* 24:5 (1988), 513–523 (see page 40).

[She24] Ming Shen. **Rethinking Data Selection for Supervised Fine-Tuning** (2024). arXiv: 2402.06094 (see page 55).

[SRB18] Vaibhav Sinha, Sukrut Rao, and Vineeth Balasubramanian. **Fast Dawid-Skene: A Fast Vote Aggregation Scheme for Sentiment Classification**. *Workshop on Issues of Sentiment Discovery and Opinion Mining (WISDOM) at the ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)* (Aug. 2018) (see page 50).

[Thi+21] Saravanan Thirumuruganathan, Han Li, Nan Tang, Mourad Ouzzani, Yash Govind, Derek Paulsen, Glenn Fung, and AnHai Doan. **Deep Learning for Blocking in Entity Matching: A Design Space Exploration**. *PVLDB* 14:11 (2021), 2459–2472 (see page 25).

[Vas+17] Ashish Vaswani, Llion Jones, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. **Attention is All you Need**. *Advances in Neural Information Processing Systems (NeurIPS)* (2017) (see page 15).

[Wan+11] Jiannan Wang, Guoliang Li, Jeffrey Xu Yu, and Jianhua Feng. **Entity Matching: How Similar is Similar**. *PVLDB* 4:10 (2011), 622–633 (see page 43).

[Win90] William E Winkler. **String Comparator Metrics and Enhanced Decision Rules in the Fellegi-Sunter Model of Record Linkage** (1990) (see page 37).

[Wu+20] Renzhi Wu, Sanya Chaba, Saurabh Sawlani, Xu Chu, and Saravanan Thirumuruganathan. **ZeroER: Entity Resolution using Zero Labeled Examples**. In: *Proceedings of the International Conference on Management of Data (SIGMOD)*. 2020, 1149–1164 (see pages 14, 27, 30, 79).

[Xu+21] Wanying Xu, Chenchen Sun, Lei Xu, Wenyu Chen, and Zhijiang Hou. **Unsupervised Entity Resolution Method Based on Random Forest**. In: *Web Information Systems and Applications*. 2021, 372–382 (see page 13).

[Zea+23a] Alexandros Zeakis, George Papadakis, Dimitrios Skoutas, and Manolis Koubarakis. **Pre-trained Embeddings for Entity Resolution: An Experimental Analysis**. *PVLDB* 16:9 (2023), 2225–2238 (see pages 2, 3, 14, 16, 23, 27, 29, 31, 77).

[Zea+23b] Alexandros Zeakis, George Papadakis, Dimitrios Skoutas, and Manolis Koubarakis. **Pre-trained Embeddings for Entity Resolution: An Experimental Analysis [Experiment, Analysis & Benchmark]** (2023). arXiv: 2304.12329 (see pages 23, 26, 28, 40, 66, 69, 79).

[Zha+22] Jieyu Zhang, Cheng-Yu Hsieh, Yue Yu, Chao Zhang, and Alexander Ratner. **A Survey on Programmatic Weak Supervision** (2022). arXiv: 2202.05433 (see page 18).